SOURCE CODE SPECIFICATION & TECHNICAL REFERENCE

Structural Beam Analysis Platform

Shear Force & Bending Moment Diagnostics (SFD & BMD) for Simply Supported, Overhanging, and Cantilever Beams

FILES DOCUMENTED	TOTAL CODE SIZE	RESOLUTION	ARCHITECTURE
6 Source Files	4,139 Lines	2,000 Sample Points	ES Modules / High-DPI Canvas

</> 1. Analytical Mechanics & Equilibrium Engine (beam-engine.js)

638 lines • JAVASCRIPT

Static equilibrium solver, continuous section mechanics evaluation across 2,000 points, root finding for zero-shear and contraflexure points, LaTeX piece-wise expressions, and CSV data export.

```
/**
 * Structural Beam Analysis Engine
 * Calculates static equilibrium, reactions, internal shear force V(x),
 * bending moment M(x), inflection points, and stress diagnostics.
 *
 * Supports:
 * - Two-support systems (Simply Supported, Single Overhang, Double Overhang)
 * - Cantilever systems (Clamped at left x=0 or right x=L or custom)
 * - Concentrated / Point loads (upward / downward)
 * - Uniformly Distributed Loads (UDL) across arbitrary sub-spans [x1, x2]
 * - Concentrated / Couple Moments (clockwise / counterclockwise)
 */

export class BeamEngine {
  constructor(config = {}) {
    this.length = config.length ?? 10; // meters
    this.supportType = config.supportType ?? 'two-supports'; // 'two-supports' | 'cantilever-left' | 'cantilever-r
    this.supportA = config.supportA ?? { x: 2, type: 'pinned' }; // x in m
    this.supportB = config.supportB ?? { x: 8, type: 'roller' }; // x in m
    this.pointLoads = config.pointLoads ?? []; // { id, x, magnitude (kN, downward > 0), label }
    this.udlLoads = config.udlLoads ?? []; // { id, x1, x2, magnitude (kN/m, downward > 0), label }
    this.moments = config.moments ?? []; // { id, x, magnitude (kN*m, clockwise > 0), label }

    // Cross-section & material parameters
    this.sectionModulus = config.sectionModulus ?? 500; // cm^3
    this.allowableStress = config.allowableStress ?? 250; // MPa (e.g. S275/A36 steel)
    this.signConvention = config.signConvention ?? 'sagging-positive'; // 'sagging-positive' (tension bottom) | 'h
  }

  // Setters
  setBeamLength(L) {
    this.length = Math.max(0.1, Number(L));
  }

  setSupports(type, posA, posB) {
    this.supportType = type;
    if (posA !== undefined) this.supportA.x = Math.max(0, Math.min(this.length, Number(posA)));
    if (posB !== undefined) this.supportB.x = Math.max(0, Math.min(this.length, Number(posB)));
  }

  addPointLoad(load) {
    const id = load.id || 'pl_' + Date.now() + '_' + Math.random().toString(36).substr(2, 4);
    this.pointLoads.push({
      id,
      x: Math.max(0, Math.min(this.length, Number(load.x))),
      magnitude: Number(load.magnitude), // downward positive (kN)
      label: load.label || `P${this.pointLoads.length + 1}`
    });
    return id;
  }

  addUDL(udl) {
    const id = udl.id || 'udl_' + Date.now() + '_' + Math.random().toString(36).substr(2, 4);
    const x1 = Math.max(0, Math.min(this.length, Number(udl.x1)));
    const x2 = Math.max(0, Math.min(this.length, Number(udl.x2)));
    this.udlLoads.push({
      id,
      x1: Math.min(x1, x2),
      x2: Math.max(x1, x2),
      magnitude: Number(udl.magnitude), // downward positive (kN/m)
      label: udl.label || `w${this.udlLoads.length + 1}`
    });
    return id;
```

```
  }

  addMoment(moment) {
    const id = moment.id || 'm_' + Date.now() + '_' + Math.random().toString(36).substr(2, 4);
    this.moments.push({
      id,
      x: Math.max(0, Math.min(this.length, Number(moment.x))),
      magnitude: Number(moment.magnitude), // clockwise positive (kN*m)
      label: moment.label || `M${this.moments.length + 1}`
    });
    return id;
  }

  removeLoad(type, id) {
    if (type === 'point') this.pointLoads = this.pointLoads.filter(l => l.id !== id);
    if (type === 'udl') this.udlLoads = this.udlLoads.filter(l => l.id !== id);
    if (type === 'moment') this.moments = this.moments.filter(l => l.id !== id);
  }

  clearLoads() {
    this.pointLoads = [];
    this.udlLoads = [];
    this.moments = [];
  }

  /**
   * Static Equilibrium Analysis
   * Computes reaction forces and reaction moments.
   */
  calculateReactions() {
    const L = this.length;
    let totalDownwardForce = 0;
    let sumMomentsAboutOrigin = 0; // Clockwise positive moments about x=0
    let steps = [];

    // Sum from point loads (downward load F at x produces clockwise moment about x=0: +F*x)
    for (const pl of this.pointLoads) {
      totalDownwardForce += pl.magnitude;
      sumMomentsAboutOrigin += pl.magnitude * pl.x;
    }

    // Sum from UDL loads
    for (const udl of this.udlLoads) {
      const span = Math.max(0, udl.x2 - udl.x1);
      const totalUDLForce = udl.magnitude * span;
      const centroid = (udl.x1 + udl.x2) / 2;
      totalDownwardForce += totalUDLForce;
      sumMomentsAboutOrigin += totalUDLForce * centroid;
    }

    // Concentrated applied moments (clockwise positive adds directly)
    for (const m of this.moments) {
      sumMomentsAboutOrigin += m.magnitude;
    }

    let reactions = {
      R1: 0, // at support A
      R2: 0, // at support B
      M_fixed: 0, // fixed clamp reaction moment if cantilever
      posA: this.supportA.x,
      posB: this.supportB.x,
      isStable: true,
      statusMessage: 'Equilibrium Satisfied',
      totalDownwardForce,
      totalUpwardReaction: 0,
      balanceError: 0,
      steps: []
    };

    if (this.supportType === 'cantilever-left') {
```

```
        // Clamped at x = 0
        // Upward reaction force R1 = total downward force
        // Counterclockwise clamp moment M_fixed balances sum of clockwise moments
        const R1 = totalDownwardForce;
        const M_fixed = -sumMomentsAboutOrigin; // Counterclockwise reaction moment
        reactions.R1 = R1;
        reactions.R2 = 0;
        reactions.M_fixed = M_fixed;
        reactions.posA = 0;
        reactions.totalUpwardReaction = R1;
        reactions.balanceError = Math.abs(R1 - totalDownwardForce);

        reactions.steps = [
          `Cantilever fixed at x = 0.00 m:`,
          `∑ Fy = 0  =>  R_fixed = Total Downward Load = ${R1.toFixed(3)} kN ↑`,
          `∑ M(x=0) = 0  =>  M_reaction = ${Math.abs(M_fixed).toFixed(3)} kN·m (${M_fixed <= 0 ? 'Counter-Clockwise'
        ];

        return reactions;
      }

      if (this.supportType === 'cantilever-right') {
        // Clamped at x = L
        const R2 = totalDownwardForce;
        // Moments about x = L:
        // A downward load at x produces counterclockwise moment about x=L: -F*(L - x)
        let sumMomentsAboutL = 0;
        for (const pl of this.pointLoads) {
          sumMomentsAboutL -= pl.magnitude * (L - pl.x);
        }
        for (const udl of this.udlLoads) {
          const span = Math.max(0, udl.x2 - udl.x1);
          const force = udl.magnitude * span;
          const centroid = (udl.x1 + udl.x2) / 2;
          sumMomentsAboutL -= force * (L - centroid);
        }
        for (const m of this.moments) {
          sumMomentsAboutL += m.magnitude;
        }

        const M_fixed = -sumMomentsAboutL;
        reactions.R1 = 0;
        reactions.R2 = R2;
        reactions.M_fixed = M_fixed;
        reactions.posB = L;
        reactions.totalUpwardReaction = R2;
        reactions.balanceError = Math.abs(R2 - totalDownwardForce);

        reactions.steps = [
          `Cantilever fixed at x = ${L.toFixed(2)} m:`,
          `∑ Fy = 0  =>  R_fixed = Total Downward Load = ${R2.toFixed(3)} kN ↑`,
          `∑ M(x=L) = 0  =>  M_reaction = ${Math.abs(M_fixed).toFixed(3)} kN·m`
        ];

        return reactions;
      }

      // Two-support system (Simply supported or Overhanging)
      const xA = this.supportA.x;
      const xB = this.supportB.x;
      const spanAB = xB - xA;

      if (Math.abs(spanAB) < 1e-4) {
        reactions.isStable = false;
        reactions.statusMessage = 'Unstable: Supports A and B coincide (Span = 0). Cannot resist overturning.';
        return reactions;
      }

      // Moment equilibrium about Support A (taking clockwise as positive):
      // R2 acts upward at xB, producing counterclockwise moment about xA: -R2 * (xB - xA)
```

```
    // Applied loads produce moment about xA:
    // Downward force at x: moment about xA = +F * (x - xA)
    let sumMomentsAboutA = 0;

    for (const pl of this.pointLoads) {
      sumMomentsAboutA += pl.magnitude * (pl.x - xA);
    }

    for (const udl of this.udlLoads) {
      const span = Math.max(0, udl.x2 - udl.x1);
      const force = udl.magnitude * span;
      const centroid = (udl.x1 + udl.x2) / 2;
      sumMomentsAboutA += force * (centroid - xA);
    }

    for (const m of this.moments) {
      sumMomentsAboutA += m.magnitude;
    }

    // ∑ M_A = 0 => sumMomentsAboutA - R2 * (xB - xA) = 0
    // => R2 = sumMomentsAboutA / (xB - xA)
    const R2 = sumMomentsAboutA / spanAB;
    const R1 = totalDownwardForce - R2;

    reactions.R1 = R1;
    reactions.R2 = R2;
    reactions.totalUpwardReaction = R1 + R2;
    reactions.balanceError = Math.abs((R1 + R2) - totalDownwardForce);

    reactions.steps = [
      `1. Equilibrium of Moments about Support A (x = ${xA.toFixed(2)} m):`,
      `   ∑ M_A = 0  =>  R_B · (${xB.toFixed(2)} - ${xA.toFixed(2)}) - (${sumMomentsAboutA.toFixed(3)} kN·m) = 0`,
      `   R_B = ${sumMomentsAboutA.toFixed(3)} / ${spanAB.toFixed(3)} = ${R2.toFixed(3)} kN ${R2 >= 0 ? '↑' : '↓'}`,
      `2. Vertical Force Equilibrium:`,
      `   ∑ F_y = 0  =>  R_A + R_B = Total Applied Load (${totalDownwardForce.toFixed(3)} kN)`,
      `   R_A = ${totalDownwardForce.toFixed(3)} - (${R2.toFixed(3)}) = ${R1.toFixed(3)} kN ${R1 >= 0 ? '↑' : '↓'}`,
      `3. Equilibrium Verification:`,
      `   |∑ F_y| = ${reactions.balanceError.toExponential(2)} kN ≈ 0.00 kN [PASS]`
    ];

    return reactions;
  }

  /**
   * Internal Section Mechanics: Evaluates V(x) and M(x) at any section x
   * Using Euler-Bernoulli left-to-right section cut:
   * Upward forces to left cause positive shear (+V)
   * Downward loads to left cause negative shear (-V)
   * Sagging moment (compression top, tension bottom) is positive:
   * - Upward forces to left cause positive moment: +R * (x - x_R)
   * - Downward loads to left cause negative moment: -P * (x - x_P)
   * - Clockwise applied moment to left causes positive bending moment
   */
  evaluateAt(x, reactions) {
    let V = 0;
    let M = 0;

    // 1. Reactions to the left of x
    if (this.supportType === 'cantilever-left') {
      if (x > 0) {
        V += reactions.R1;
        M += reactions.R1 * x + reactions.M_fixed;
      }
    } else if (this.supportType === 'cantilever-right') {
      // Right cantilever: reaction is at x=L, so no reaction to left if x < L
      // Only applied loads to the left act
    } else {
      // Two supports
      if (x > reactions.posA) {
        V += reactions.R1;
```

```
            M += reactions.R1 * (x - reactions.posA);
          }
          if (x > reactions.posB) {
            V += reactions.R2;
            M += reactions.R2 * (x - reactions.posB);
          }
        }

        // 2. Point loads to the left of x
        for (const pl of this.pointLoads) {
          if (x > pl.x) {
            V -= pl.magnitude;
            M -= pl.magnitude * (x - pl.x);
          }
        }

        // 3. UDL loads to the left of x
        for (const udl of this.udlLoads) {
          if (x > udl.x1) {
            const activeSpan = Math.min(x, udl.x2) - udl.x1;
            if (activeSpan > 0) {
              const force = udl.magnitude * activeSpan;
              const centroid = udl.x1 + activeSpan / 2;
              V -= force;
              M -= force * (x - centroid);
            }
          }
        }

        // 4. Concentrated moments to the left of x
        for (const m of this.moments) {
          if (x > m.x) {
            M += m.magnitude;
          }
        }

        // Flip moment sign if civil/hogging-positive convention is selected
        const signFactor = this.signConvention === 'hogging-positive' ? -1 : 1;

        return {
          x,
          V,
          M: M * signFactor
        };
      }

      /**
       * Continuous Analysis over 2000 points with exact step discontinuity capture
       */
      solve() {
        const reactions = this.calculateReactions();
        const L = this.length;
        const numPoints = 2000;

        // Collect all critical discontinuity boundaries
        const criticalX = new Set([0, L]);
        if (this.supportType === 'cantilever-left') {
          criticalX.add(0);
        } else if (this.supportType === 'cantilever-right') {
          criticalX.add(L);
        } else {
          criticalX.add(this.supportA.x);
          criticalX.add(this.supportB.x);
        }

        for (const pl of this.pointLoads) criticalX.add(pl.x);
        for (const udl of this.udlLoads) {
          criticalX.add(udl.x1);
          criticalX.add(udl.x2);
        }
```

```
    for (const m of this.moments) criticalX.add(m.x);

    const sortedCrit = Array.from(criticalX).sort((a, b) => a - b);

    // Build sampling x-points list including exact dual points (x - eps, x + eps)
    const eps = 1e-6;
    const xSamplePoints = [];

    // Uniform grid
    const dx = L / (numPoints - 1);
    for (let i = 0; i < numPoints; i++) {
      xSamplePoints.push(i * dx);
    }

    // Inject discontinuity pairs
    for (const cx of sortedCrit) {
      if (cx > 0 && cx < L) {
        xSamplePoints.push(Math.max(0, cx - eps));
        xSamplePoints.push(Math.min(L, cx + eps));
      } else if (cx === 0) {
        xSamplePoints.push(eps);
      } else if (cx === L) {
        xSamplePoints.push(L - eps);
      }
    }

    // Sort and remove duplicates within 1e-9
    xSamplePoints.sort((a, b) => a - b);
    const uniqueX = [];
    for (let i = 0; i < xSamplePoints.length; i++) {
      if (i === 0 || Math.abs(xSamplePoints[i] - xSamplePoints[i - 1]) > 1e-7) {
        uniqueX.push(xSamplePoints[i]);
      }
    }

    // Compute V(x) and M(x) across points
    const points = [];
    let maxV = -Infinity;
    let minV = Infinity;
    let maxAbsV = 0;
    let maxM = -Infinity;
    let minM = Infinity;
    let maxAbsM = 0;
    let maxVPos = 0;
    let maxMPos = 0;
    let minMPos = 0;

    for (const x of uniqueX) {
      const res = this.evaluateAt(x, reactions);
      points.push(res);

      if (res.V > maxV) maxV = res.V;
      if (res.V < minV) minV = res.V;
      if (Math.abs(res.V) > maxAbsV) {
        maxAbsV = Math.abs(res.V);
        maxVPos = res.x;
      }

      if (res.M > maxM) {
        maxM = res.M;
        maxMPos = res.x;
      }
      if (res.M < minM) {
        minM = res.M;
        minMPos = res.x;
      }
      if (Math.abs(res.M) > maxAbsM) {
        maxAbsM = Math.abs(res.M);
      }
    }
```

```
414
415       // Detect Points of Zero Shear (where V(x) crosses zero or jumps across zero)
416       const zeroShearPoints = this.findZeroShearPoints(points, sortedCrit, reactions);
417
418       // Detect Points of Contraflexure / Inflection (where M(x) = 0)
419       const contraflexurePoints = this.findContraflexurePoints(points, sortedCrit, reactions);
420
421       // Compute maximum bending stress σ = |M_max| / Z
422       // M in kN*m = 10^6 N*mm
423       // Z in cm^3 = 10^3 mm^3
424       // σ = (M * 10^6 N*mm) / (Z * 10^3 mm^3) = (M / Z) * 1000 MPa (N/mm^2)
425       const peakMomentAbs = Math.max(Math.abs(maxM), Math.abs(minM));
426       const peakStress = this.sectionModulus > 0 ? (peakMomentAbs * 1000) / this.sectionModulus : 0; // MPa
427       const stressRatio = this.allowableStress > 0 ? (peakStress / this.allowableStress) * 100 : 0; // %
428       const safetyFactor = peakStress > 0 ? this.allowableStress / peakStress : Infinity;
429
430       return {
431         reactions,
432         points,
433         sortedCrit,
434         diagnostics: {
435           maxV: Number(maxV.toFixed(3)),
436           minV: Number(minV.toFixed(3)),
437           maxAbsV: Number(maxAbsV.toFixed(3)),
438           maxVPos: Number(maxVPos.toFixed(3)),
439           maxM: Number(maxM.toFixed(3)),
440           maxMPos: Number(maxMPos.toFixed(3)),
441           minM: Number(minM.toFixed(3)),
442           minMPos: Number(minMPos.toFixed(3)),
443           maxAbsM: Number(maxAbsM.toFixed(3)),
444           peakStress: Number(peakStress.toFixed(2)),
445           stressRatio: Number(stressRatio.toFixed(1)),
446           safetyFactor: Number(isFinite(safetyFactor) ? safetyFactor.toFixed(2) : '99.9'),
447           zeroShearPoints,
448           contraflexurePoints,
449           totalLoad: Number(reactions.totalDownwardForce.toFixed(3)),
450           totalReaction: Number(reactions.totalUpwardReaction.toFixed(3)),
451           balanceCheck: reactions.balanceError < 1e-4
452         }
453       };
454     }
455
456     /**
457      * Find all points where Shear Force V(x) = 0
458      * These correspond to local extrema of Bending Moment M(x)
459      * Captures both continuous slope crossings inside UDLs and step jumps across zero at point loads/reactions
460      */
461     findZeroShearPoints(points, boundaries, reactions) {
462       const results = [];
463       if (this.pointLoads.length === 0 && this.udlLoads.length === 0 && this.moments.length === 0) {
464         return results;
465       }
466
467       const eps = 1e-5;
468
469       // 1. Check for step jumps across zero at boundary points (point loads & reactions)
470       for (const bx of boundaries) {
471         if (bx <= 0.01 || bx >= this.length - 0.01) continue;
472
473         const evalBefore = this.evaluateAt(bx - eps, reactions);
474         const evalAfter = this.evaluateAt(bx + eps, reactions);
475
476         if (evalBefore.V * evalAfter.V <= 0 && Math.abs(evalAfter.V - evalBefore.V) > 0.05) {
477           const rootEval = this.evaluateAt(bx, reactions);
478           if (!results.some(r => Math.abs(r.x - bx) < 0.05)) {
479             results.push({
480               x: Number(bx.toFixed(3)),
481               V: 0,
482               M: Number(Math.max(Math.abs(evalBefore.M), Math.abs(evalAfter.M)) * (evalBefore.M >= 0 ? 1 : -1)).toFi
483               type: 'Zero Shear Step / Peak Moment'
```

```
            });
          }
        }
      }

      // 2. Check for continuous zero crossings inside spans between boundaries
      for (let i = 0; i < boundaries.length - 1; i++) {
        const xA = boundaries[i];
        const xB = boundaries[i + 1];
        if (xB - xA < 1e-4) continue;

        const evalA = this.evaluateAt(xA + eps, reactions);
        const evalB = this.evaluateAt(xB - eps, reactions);

        if (evalA.V * evalB.V < -1e-5) {
          const dV = evalB.V - evalA.V;
          let rootX;
          if (Math.abs(dV) > 1e-6) {
            rootX = xA + (-evalA.V / dV) * (xB - xA);
          } else {
            rootX = (xA + xB) / 2;
          }

          rootX = Math.max(xA, Math.min(xB, rootX));
          const rootEval = this.evaluateAt(rootX, reactions);

          if (!results.some(r => Math.abs(r.x - rootX) < 0.05)) {
            results.push({
              x: Number(rootX.toFixed(3)),
              V: 0,
              M: Number(rootEval.M.toFixed(3)),
              type: 'Zero Shear / Peak Moment'
            });
          }
        }
      }

      results.sort((a, b) => a.x - b.x);
      return results;
    }

    /**
     * Find Points of Contraflexure (where Bending Moment M(x) = 0)
     * Inflection points between sagging and hogging zones (curvature changes sign)
     */
    findContraflexurePoints(points, boundaries, reactions) {
      const results = [];
      if (this.pointLoads.length === 0 && this.udlLoads.length === 0 && this.moments.length === 0) {
        return results;
      }

      for (let i = 0; i < points.length - 1; i++) {
        const p1 = points[i];
        const p2 = points[i + 1];

        // Exclude boundary zeros at free beam ends
        if (p1.x < 0.05) continue;
        if (p2.x > this.length - 0.05) continue;

        // Strict sign crossing: p1.M * p2.M < 0
        if (p1.M * p2.M < -1e-6) {
          let rootX = p1.x;
          const dM = p2.M - p1.M;
          if (Math.abs(dM) > 1e-6) {
            rootX = p1.x + (-p1.M / dM) * (p2.x - p1.x);
          }
          rootX = Number(rootX.toFixed(3));

          if (rootX > 0.05 && rootX < this.length - 0.05 && !results.some(r => Math.abs(r.x - rootX) < 0.08)) {
            const evalAtRoot = this.evaluateAt(rootX, reactions);
```

```
            results.push({
              x: rootX,
              V: Number(evalAtRoot.V.toFixed(3)),
              M: 0,
              type: 'Point of Contraflexure'
            });
          }
        }
      }

      return results;
    }

    /**
     * Generate Piece-wise LaTeX equations for V(x) and M(x)
     */
    generateLatexEquations(reactions) {
      const boundaries = Array.from(new Set([
        0,
        this.supportA.x,
        this.supportB.x,
        ...this.pointLoads.map(p => p.x),
        ...this.udlLoads.flatMap(u => [u.x1, u.x2]),
        ...this.moments.map(m => m.x),
        this.length
      ])).sort((a, b) => a - b);

      let vLatex = `\\begin{aligned}\n`;
      vLatex += `V(x) = \\begin{cases}\n`;

      let mLatex = `\\begin{aligned}\n`;
      mLatex += `M(x) = \\begin{cases}\n`;

      for (let i = 0; i < boundaries.length - 1; i++) {
        const x1 = boundaries[i];
        const x2 = boundaries[i + 1];
        if (x2 - x1 < 1e-4) continue;

        const mid = (x1 + x2) / 2;
        const evalMid = this.evaluateAt(mid, reactions);
        const evalStart = this.evaluateAt(x1 + 1e-5, reactions);
        const evalEnd = this.evaluateAt(x2 - 1e-5, reactions);

        const dV = (evalEnd.V - evalStart.V) / (x2 - x1);

        // V(x) expression
        let vExpr = '';
        if (Math.abs(dV) < 1e-3) {
          vExpr = `${evalMid.V.toFixed(2)}`;
        } else {
          const slope = dV.toFixed(2);
          const intercept = (evalStart.V - dV * x1).toFixed(2);
          vExpr = `${slope}x ${Number(intercept) >= 0 ? '+' : '-'} ${Math.abs(Number(intercept))}`;
        }

        vLatex += `  ${vExpr}, & ${x1.toFixed(2)} \\le x < ${x2.toFixed(2)} \\\\[4pt]\n`;

        // M(x) approximation for display
        let mExpr = `\\int V(x)dx = ${evalMid.M.toFixed(2)}`;
        mLatex += `  ${evalStart.M.toFixed(2)} \\to ${evalEnd.M.toFixed(2)}, & ${x1.toFixed(2)} \\le x < ${x2.toFixe
      }

      vLatex += `\\end{cases}\n\\end{aligned}`;
      mLatex += `\\end{cases}\n\\end{aligned}`;

      return {
        shearLatex: vLatex,
        momentLatex: mLatex,
        reactionsLatex: `R_A = ${reactions.R1.toFixed(2)}\\,\\text{kN},\\quad R_B = ${reactions.R2.toFixed(2)}\\,\\t
      };
```

```
    }

    /**
     * Export CSV of computed points
     */
    exportCSV(solution) {
      let csv = `x_m,ShearForce_V_kN,BendingMoment_M_kNm,BendingStress_sigma_MPa\n`;
      for (const pt of solution.points) {
        const stress = this.sectionModulus > 0 ? (Math.abs(pt.M) * 1000) / this.sectionModulus : 0;
        csv += `${pt.x.toFixed(4)},${pt.V.toFixed(4)},${pt.M.toFixed(4)},${stress.toFixed(2)}\n`;
      }
      return csv;
    }
  }

```

</> 2. High-DPI Canvas Multi-Diagram Renderer (beam-renderer.js)

985 lines • JAVASCRIPT

HiDPI synchronized canvas rendering for 2D Beam Schematic, Shear Force Diagram (SFD), Bending Moment Diagram (BMD), and synchronized crosshair scrubbing.

```
/**
 * High-DPI Canvas Renderer for:
 * 1. Beam Schematic Diagram (Beam, Supports, Reactions, Point Loads, UDLs, Moments)
 * 2. Shear Force Diagram (SFD)
 * 3. Bending Moment Diagram (BMD)
 * With Linked Synchronized Crosshairs & Floating Diagnostic HUD
 */

export class BeamRenderer {
  constructor(canvases, options = {}) {
    this.beamCanvas = canvases.beam;
    this.sfdCanvas = canvases.sfd;
    this.bmdCanvas = canvases.bmd;

    this.beamCtx = this.beamCanvas.getContext('2d');
    this.sfdCtx = this.sfdCanvas.getContext('2d');
    this.bmdCtx = this.bmdCanvas.getContext('2d');

    this.isDarkMode = options.isDarkMode ?? true;
    this.hoverX = null; // in meters along beam
    this.activeCanvas = null;

    this.padding = { left: 75, right: 40, top: 50, bottom: 40 };

    this.initEvents();
  }

  setTheme(isDark) {
    this.isDarkMode = isDark;
  }

  resizeCanvas(canvas, ctx) {
    const rect = canvas.getBoundingClientRect();
    const dpr = window.devicePixelRatio || 1;

    const displayW = Math.max(100, Math.round(rect.width || canvas.clientWidth || (canvas.parentElement ? canvas.p
    const displayH = Math.max(80, Math.round(rect.height || canvas.clientHeight || (canvas.parentElement ? canvas.

    const targetW = Math.round(displayW * dpr);
    const targetH = Math.round(displayH * dpr);

    if (canvas.width !== targetW || canvas.height !== targetH) {
      canvas.width = targetW;
      canvas.height = targetH;
    }

    ctx.setTransform(dpr, 0, 0, dpr, 0, 0);

    return {
      width: displayW,
      height: displayH,
      dpr
    };
  }

  initEvents() {
    const canvases = [this.beamCanvas, this.sfdCanvas, this.bmdCanvas];

    canvases.forEach(canvas => {
      const handlePointer = (e) => {
        const rect = canvas.getBoundingClientRect();
        const clientX = e.clientX || (e.touches && e.touches[0] ? e.touches[0].clientX : null);
        if (clientX === null) return;
```

```
64
65          const px = clientX - rect.left;
66          const beamW = rect.width - this.padding.left - this.padding.right;
67
68          if (px >= this.padding.left && px <= rect.width - this.padding.right) {
69            const ratio = (px - this.padding.left) / beamW;
70            this.hoverX = Math.max(0, Math.min(this.engine.length, ratio * this.engine.length));
71          } else if (px < this.padding.left) {
72            this.hoverX = 0;
73          } else {
74            this.hoverX = this.engine.length;
75          }
76
77          this.renderAll();
78          if (this.onInspectCallback) {
79            this.onInspectCallback(this.hoverX, e.clientX, e.clientY);
80          }
81        };
82
83        canvas.addEventListener('mousemove', handlePointer);
84        canvas.addEventListener('touchmove', handlePointer, { passive: true });
85
86        const handleLeave = () => {
87          this.hoverX = null;
88          this.renderAll();
89          if (this.onInspectCallback) {
90            this.onInspectCallback(null);
91          }
92        };
93
94        canvas.addEventListener('mouseleave', handleLeave);
95        canvas.addEventListener('touchend', handleLeave);
96      });
97    }
98
99    onInspect(callback) {
100     this.onInspectCallback = callback;
101   }
102
103   render(engine, solution) {
104     this.engine = engine;
105     this.solution = solution;
106     this.renderAll();
107   }
108
109   renderAll() {
110     if (!this.engine || !this.solution) return;
111     if (this._rafId) cancelAnimationFrame(this._rafId);
112     this._rafId = requestAnimationFrame(() => {
113       this.renderBeamSchematic();
114       this.renderSFD();
115       this.renderBMD();
116     });
117   }
118
119   getColors() {
120     if (this.isDarkMode) {
121       return {
122         bg: '#0f172a',
123         cardBg: '#1e293b',
124         grid: 'rgba(255, 255, 255, 0.07)',
125         axis: '#64748b',
126         text: '#f8fafc',
127         textMuted: '#94a3b8',
128         beamBody: '#475569',
129         beamFlange: '#334155',
130         support: '#94a3b8',
131         reaction: '#10b981', // emerald
132         pointLoad: '#f59e0b', // amber
133         udl: '#06b6d4', // cyan
```

```
        moment: '#a855f7', // purple
        sfdLine: '#06b6d4',
        sfdFillPos: 'rgba(6, 182, 212, 0.35)',
        sfdFillNeg: 'rgba(244, 63, 94, 0.35)',
        bmdLine: '#6366f1',
        bmdFillSag: 'rgba(16, 185, 129, 0.35)',
        bmdFillHog: 'rgba(168, 85, 247, 0.35)',
        crosshair: '#e2e8f0',
        crosshairGlow: 'rgba(99, 102, 241, 0.6)',
        highlight: '#38bdf8'
      };
    } else {
      return {
        bg: '#ffffff',
        cardBg: '#f8fafc',
        grid: 'rgba(0, 0, 0, 0.06)',
        axis: '#94a3b8',
        text: '#0f172a',
        textMuted: '#64748b',
        beamBody: '#cbd5e1',
        beamFlange: '#94a3b8',
        support: '#475569',
        reaction: '#059669',
        pointLoad: '#d97706',
        udl: '#0891b2',
        moment: '#9333ea',
        sfdLine: '#0891b2',
        sfdFillPos: 'rgba(8, 145, 178, 0.25)',
        sfdFillNeg: 'rgba(225, 29, 72, 0.25)',
        bmdLine: '#4f46e5',
        bmdFillSag: 'rgba(5, 150, 105, 0.25)',
        bmdFillHog: 'rgba(147, 51, 234, 0.25)',
        crosshair: '#0f172a',
        crosshairGlow: 'rgba(79, 70, 229, 0.4)',
        highlight: '#0284c7'
      };
    }
  }

  // Conversion: x in meters to canvas px
  metersToPx(x, beamWidth) {
    return this.padding.left + (x / this.engine.length) * beamWidth;
  }

  /**
   * 1. BEAM SCHEMATIC RENDERING
   */
  renderBeamSchematic() {
    const ctx = this.beamCtx;
    const { width, height } = this.resizeCanvas(this.beamCanvas, ctx);
    const colors = this.getColors();
    const beamW = width - this.padding.left - this.padding.right;
    const beamY = height * 0.58; // beam centerline
    const beamThickness = 14;

    ctx.clearRect(0, 0, width, height);

    // Dimension baseline and tick markers
    ctx.strokeStyle = colors.grid;
    ctx.lineWidth = 1;
    const numTicks = Math.min(10, Math.ceil(this.engine.length));
    for (let i = 0; i <= numTicks; i++) {
      const xVal = (i / numTicks) * this.engine.length;
      const px = this.metersToPx(xVal, beamW);

      ctx.beginPath();
      ctx.moveTo(px, this.padding.top - 10);
      ctx.lineTo(px, height - 15);
      ctx.stroke();

```

```
        ctx.fillStyle = colors.textMuted;
        ctx.font = '11px "JetBrains Mono", monospace';
        ctx.textAlign = 'center';
        ctx.fillText(`${xVal.toFixed(1)}m`, px, height - 4);
      }

      // Draw UDL Blocks (rendered above beam)
      for (const udl of this.engine.udlLoads) {
        this.drawUDL(ctx, udl, beamW, beamY - beamThickness / 2, colors);
      }

      // Draw Beam Member (I-Beam silhouette with 3D bevel)
      const beamStartPx = this.metersToPx(0, beamW);
      const beamEndPx = this.metersToPx(this.engine.length, beamW);

      // Beam drop shadow
      ctx.shadowColor = 'rgba(0, 0, 0, 0.25)';
      ctx.shadowBlur = 8;
      ctx.shadowOffsetY = 3;

      ctx.fillStyle = colors.beamBody;
      ctx.beginPath();
      ctx.roundRect?.(beamStartPx, beamY - beamThickness / 2, beamEndPx - beamStartPx, beamThickness, 3);
      ctx.fill();

      ctx.shadowColor = 'transparent';

      // Flange top and bottom borders
      ctx.strokeStyle = colors.beamFlange;
      ctx.lineWidth = 2;
      ctx.beginPath();
      ctx.moveTo(beamStartPx, beamY - beamThickness / 2);
      ctx.lineTo(beamEndPx, beamY - beamThickness / 2);
      ctx.moveTo(beamStartPx, beamY + beamThickness / 2);
      ctx.lineTo(beamEndPx, beamY + beamThickness / 2);
      ctx.stroke();

      // Centerline dashed guide
      ctx.strokeStyle = 'rgba(255, 255, 255, 0.2)';
      ctx.setLineDash([4, 4]);
      ctx.beginPath();
      ctx.moveTo(beamStartPx, beamY);
      ctx.lineTo(beamEndPx, beamY);
      ctx.stroke();
      ctx.setLineDash([]);

      // Draw Supports
      this.drawSupports(ctx, beamW, beamY + beamThickness / 2, colors);

      // Draw Reactions
      this.drawReactions(ctx, beamW, beamY + beamThickness / 2, colors);

      // Draw Point Loads
      for (const pl of this.engine.pointLoads) {
        this.drawPointLoad(ctx, pl, beamW, beamY - beamThickness / 2, colors);
      }

      // Draw Applied Concentrated Moments
      for (const m of this.engine.moments) {
        this.drawMoment(ctx, m, beamW, beamY, colors);
      }

      // Synchronized Crosshair on Beam
      if (this.hoverX !== null) {
        const hx = this.metersToPx(this.hoverX, beamW);
        this.drawCrosshairLine(ctx, hx, 0, height, colors);
      }
    }

    drawSupports(ctx, beamW, yPos, colors) {
```

```
    const type = this.engine.supportType;

    if (type === 'cantilever-left') {
      const x = this.metersToPx(0, beamW);
      this.drawFixedSupport(ctx, x, yPos - 7, 'left', colors);
      return;
    }
    if (type === 'cantilever-right') {
      const x = this.metersToPx(this.engine.length, beamW);
      this.drawFixedSupport(ctx, x, yPos - 7, 'right', colors);
      return;
    }

    // Two supports: Support A & B
    const posA = this.metersToPx(this.engine.supportA.x, beamW);
    const posB = this.metersToPx(this.engine.supportB.x, beamW);

    this.drawPinSupport(ctx, posA, yPos, 'Support A (Pinned)', colors);
    this.drawRollerSupport(ctx, posB, yPos, 'Support B (Roller)', colors);
  }

  drawPinSupport(ctx, x, y, label, colors) {
    const size = 16;
    ctx.save();

    // Pin pivot dot
    ctx.fillStyle = colors.text;
    ctx.beginPath();
    ctx.arc(x, y + 2, 3.5, 0, Math.PI * 2);
    ctx.fill();

    // Triangle
    ctx.fillStyle = colors.support;
    ctx.strokeStyle = colors.axis;
    ctx.lineWidth = 1.5;
    ctx.beginPath();
    ctx.moveTo(x, y + 2);
    ctx.lineTo(x - size * 0.7, y + size + 2);
    ctx.lineTo(x + size * 0.7, y + size + 2);
    ctx.closePath();
    ctx.fill();
    ctx.stroke();

    // Ground hatching
    ctx.strokeStyle = colors.axis;
    ctx.lineWidth = 1.5;
    ctx.beginPath();
    ctx.moveTo(x - size, y + size + 4);
    ctx.lineTo(x + size, y + size + 4);
    ctx.stroke();

    for (let i = -size; i <= size; i += 6) {
      ctx.beginPath();
      ctx.moveTo(x + i, y + size + 4);
      ctx.lineTo(x + i - 4, y + size + 10);
      ctx.stroke();
    }

    ctx.restore();
  }

  drawRollerSupport(ctx, x, y, label, colors) {
    const size = 16;
    ctx.save();

    // Pivot dot
    ctx.fillStyle = colors.text;
    ctx.beginPath();
    ctx.arc(x, y + 2, 3.5, 0, Math.PI * 2);
    ctx.fill();
```

```
344
345       // Triangle
346       ctx.fillStyle = colors.support;
347       ctx.strokeStyle = colors.axis;
348       ctx.lineWidth = 1.5;
349       ctx.beginPath();
350       ctx.moveTo(x, y + 2);
351       ctx.lineTo(x - size * 0.7, y + size);
352       ctx.lineTo(x + size * 0.7, y + size);
353       ctx.closePath();
354       ctx.fill();
355       ctx.stroke();
356
357       // Roller wheels (2 circles)
358       ctx.fillStyle = colors.cardBg;
359       ctx.strokeStyle = colors.axis;
360       ctx.lineWidth = 1.5;
361
362       ctx.beginPath();
363       ctx.arc(x - 5, y + size + 4, 3, 0, Math.PI * 2);
364       ctx.arc(x + 5, y + size + 4, 3, 0, Math.PI * 2);
365       ctx.fill();
366       ctx.stroke();
367
368       // Ground line
369       ctx.beginPath();
370       ctx.moveTo(x - size, y + size + 8);
371       ctx.lineTo(x + size, y + size + 8);
372       ctx.stroke();
373
374       ctx.restore();
375     }
376
377     drawFixedSupport(ctx, x, y, side, colors) {
378       const h = 48;
379       const w = 12;
380       ctx.save();
381
382       ctx.fillStyle = colors.support;
383       ctx.strokeStyle = colors.axis;
384       ctx.lineWidth = 2;
385
386       const leftX = side === 'left' ? x - w : x;
387       ctx.fillRect(leftX, y - h / 2, w, h);
388       ctx.strokeRect(leftX, y - h / 2, w, h);
389
390       // Hatch lines
391       ctx.lineWidth = 1.5;
392       for (let dy = -h / 2; dy <= h / 2; dy += 8) {
393         ctx.beginPath();
394         if (side === 'left') {
395           ctx.moveTo(x - w, y + dy);
396           ctx.lineTo(x - w - 6, y + dy + 6);
397         } else {
398           ctx.moveTo(x + w, y + dy);
399           ctx.lineTo(x + w + 6, y + dy + 6);
400         }
401         ctx.stroke();
402       }
403
404       ctx.restore();
405     }
406
407     drawReactions(ctx, beamW, yPos, colors) {
408       const r = this.solution.reactions;
409       if (!r || !r.isStable) return;
410
411       if (this.engine.supportType === 'two-supports') {
412         this.drawReactionVector(ctx, this.metersToPx(r.posA, beamW), yPos + 22, r.R1, 'R_A', colors);
413         this.drawReactionVector(ctx, this.metersToPx(r.posB, beamW), yPos + 22, r.R2, 'R_B', colors);
```

```
      } else if (this.engine.supportType === 'cantilever-left') {
        this.drawReactionVector(ctx, this.metersToPx(0, beamW), yPos + 18, r.R1, 'R_fixed', colors);
      } else if (this.engine.supportType === 'cantilever-right') {
        this.drawReactionVector(ctx, this.metersToPx(this.engine.length, beamW), yPos + 18, r.R2, 'R_fixed', colors)
      }
    }

    drawReactionVector(ctx, x, y, mag, name, colors) {
      ctx.save();
      const arrowLen = 32;
      const isUpward = mag >= 0;
      const color = colors.reaction;

      ctx.strokeStyle = color;
      ctx.fillStyle = color;
      ctx.lineWidth = 2.5;

      // Arrow line
      const yStart = isUpward ? y + arrowLen : y - arrowLen;
      const yEnd = y;

      ctx.beginPath();
      ctx.moveTo(x, yStart);
      ctx.lineTo(x, yEnd);
      ctx.stroke();

      // Arrowhead
      const tipDir = isUpward ? -1 : 1;
      ctx.beginPath();
      ctx.moveTo(x, yEnd);
      ctx.lineTo(x - 5, yEnd - tipDir * 8);
      ctx.lineTo(x + 5, yEnd - tipDir * 8);
      ctx.closePath();
      ctx.fill();

      // Text badge with reaction magnitude
      ctx.font = 'bold 11px "JetBrains Mono", monospace';
      ctx.textAlign = 'center';
      const textY = isUpward ? yStart + 13 : yStart - 6;
      ctx.fillText(`${name}: ${Math.abs(mag).toFixed(1)} kN`, x, textY);

      ctx.restore();
    }

    drawPointLoad(ctx, pl, beamW, yPos, colors) {
      const x = this.metersToPx(pl.x, beamW);
      const arrowLen = Math.min(55, Math.max(28, 22 + Math.abs(pl.magnitude) * 0.6));
      const isDown = pl.magnitude >= 0;
      const color = colors.pointLoad;

      ctx.save();
      ctx.strokeStyle = color;
      ctx.fillStyle = color;
      ctx.lineWidth = 2.5;

      const yStart = isDown ? yPos - arrowLen : yPos + arrowLen;
      const yEnd = yPos;

      ctx.beginPath();
      ctx.moveTo(x, yStart);
      ctx.lineTo(x, yEnd);
      ctx.stroke();

      // Arrowhead
      const tipDir = isDown ? 1 : -1;
      ctx.beginPath();
      ctx.moveTo(x, yEnd);
      ctx.lineTo(x - 5, yEnd - tipDir * 9);
      ctx.lineTo(x + 5, yEnd - tipDir * 9);
      ctx.closePath();
```

```
      ctx.fill();

      // Load Label Pill
      const labelText = `${pl.label ? pl.label + ': ' : ''}${Math.abs(pl.magnitude)} kN`;
      ctx.font = 'bold 10px "Inter", sans-serif';
      const textMetrics = ctx.measureText(labelText);
      const pillW = textMetrics.width + 12;
      const pillH = 18;
      const pillY = isDown ? yStart - pillH - 2 : yStart + 2;

      ctx.fillStyle = colors.cardBg;
      ctx.strokeStyle = color;
      ctx.lineWidth = 1;
      ctx.beginPath();
      ctx.roundRect?.(x - pillW / 2, pillY, pillW, pillH, 4);
      ctx.fill();
      ctx.stroke();

      ctx.fillStyle = color;
      ctx.textAlign = 'center';
      ctx.fillText(labelText, x, pillY + 12);

      ctx.restore();
    }

    drawUDL(ctx, udl, beamW, yPos, colors) {
      const x1 = this.metersToPx(udl.x1, beamW);
      const x2 = this.metersToPx(udl.x2, beamW);
      const spanW = x2 - x1;
      if (spanW <= 0) return;

      const blockH = Math.min(36, Math.max(18, 14 + Math.abs(udl.magnitude) * 0.8));
      const color = colors.udl;

      ctx.save();

      // Shaded box above beam
      ctx.fillStyle = this.isDarkMode ? 'rgba(6, 182, 212, 0.15)' : 'rgba(8, 145, 178, 0.15)';
      ctx.fillRect(x1, yPos - blockH, spanW, blockH);

      // Box top line
      ctx.strokeStyle = color;
      ctx.lineWidth = 1.5;
      ctx.beginPath();
      ctx.moveTo(x1, yPos - blockH);
      ctx.lineTo(x2, yPos - blockH);
      ctx.stroke();

      // Array of small downward arrows
      const arrowSpacing = Math.max(14, Math.min(24, spanW / 12));
      const count = Math.floor(spanW / arrowSpacing);

      ctx.fillStyle = color;
      for (let i = 0; i <= count; i++) {
        const ax = x1 + (i / count) * spanW;
        ctx.beginPath();
        ctx.moveTo(ax, yPos - blockH);
        ctx.lineTo(ax, yPos);
        ctx.stroke();

        // Arrowhead at bottom touching beam
        ctx.beginPath();
        ctx.moveTo(ax, yPos);
        ctx.lineTo(ax - 3.5, yPos - 6);
        ctx.lineTo(ax + 3.5, yPos - 6);
        ctx.closePath();
        ctx.fill();
      }

      // Label on top
```

```
      const labelText = `${udl.label ? udl.label + ': ' : ''}${Math.abs(udl.magnitude)} kN/m`;
      ctx.font = 'bold 10px "Inter", sans-serif';
      ctx.textAlign = 'center';

      const midX = (x1 + x2) / 2;
      ctx.fillStyle = colors.cardBg;
      const tw = ctx.measureText(labelText).width + 10;
      ctx.fillRect(midX - tw / 2, yPos - blockH - 14, tw, 14);
      ctx.strokeStyle = color;
      ctx.strokeRect(midX - tw / 2, yPos - blockH - 14, tw, 14);

      ctx.fillStyle = color;
      ctx.fillText(labelText, midX, yPos - blockH - 3);

      ctx.restore();
    }

    drawMoment(ctx, m, beamW, yPos, colors) {
      const x = this.metersToPx(m.x, beamW);
      const radius = 18;
      const isCW = m.magnitude >= 0;
      const color = colors.moment;

      ctx.save();
      ctx.strokeStyle = color;
      ctx.lineWidth = 2.2;

      // Curved circular arc
      ctx.beginPath();
      const startAngle = isCW ? Math.PI * 0.2 : Math.PI * 1.8;
      const endAngle = isCW ? Math.PI * 1.6 : Math.PI * 0.4;
      ctx.arc(x, yPos - 12, radius, startAngle, endAngle, !isCW);
      ctx.stroke();

      // Arrowhead on arc tip
      const tipAngle = endAngle;
      const tipX = x + radius * Math.cos(tipAngle);
      const tipY = (yPos - 12) + radius * Math.sin(tipAngle);

      ctx.fillStyle = color;
      ctx.beginPath();
      ctx.arc(tipX, tipY, 4, 0, Math.PI * 2);
      ctx.fill();

      // Text label
      const labelText = `${m.label ? m.label + ': ' : ''}${Math.abs(m.magnitude)} kN·m ${isCW ? '↻' : '↺'}`;
      ctx.font = 'bold 10px "Inter", sans-serif';
      ctx.textAlign = 'center';
      ctx.fillStyle = color;
      ctx.fillText(labelText, x, yPos - radius - 16);

      ctx.restore();
    }

    /**
     * 2. SHEAR FORCE DIAGRAM (SFD) RENDERING
     */
    renderSFD() {
      const ctx = this.sfdCtx;
      const { width, height } = this.resizeCanvas(this.sfdCanvas, ctx);
      const colors = this.getColors();
      const beamW = width - this.padding.left - this.padding.right;
      const plotH = height - this.padding.top - this.padding.bottom;

      ctx.clearRect(0, 0, width, height);

      const points = this.solution.points;
      if (!points || points.length === 0) return;

      // Determine scale: max absolute V with padding
```

```
        const maxAbsV = Math.max(1, this.solution.diagnostics.maxAbsV * 1.25);
        const zeroY = this.padding.top + plotH / 2;

        // Draw Grid and Axis
        this.drawDiagramGrid(ctx, width, height, beamW, plotH, zeroY, maxAbsV, 'kN', colors);

        // Draw SFD Polygons (Separated by positive & negative fills)
        ctx.save();
        ctx.lineWidth = 2;

        // Path for gradient fills
        // Positive Fill (Cyan/Blue gradient)
        const posGrad = ctx.createLinearGradient(0, this.padding.top, 0, zeroY);
        posGrad.addColorStop(0, colors.sfdFillPos);
        posGrad.addColorStop(1, 'rgba(6, 182, 212, 0.02)');

        // Negative Fill (Rose/Amber gradient)
        const negGrad = ctx.createLinearGradient(0, zeroY, 0, height - this.padding.bottom);
        negGrad.addColorStop(0, 'rgba(244, 63, 94, 0.02)');
        negGrad.addColorStop(1, colors.sfdFillNeg);

        // 1. Positive Region Fill (Single Batched Draw Call with Clip to upper half)
        ctx.save();
        ctx.beginPath();
        ctx.rect(this.padding.left, this.padding.top, beamW, plotH / 2);
        ctx.clip();

        ctx.fillStyle = posGrad;
        ctx.beginPath();
        ctx.moveTo(this.metersToPx(points[0].x, beamW), zeroY);
        for (let i = 0; i < points.length; i++) {
          ctx.lineTo(this.metersToPx(points[i].x, beamW), zeroY - (points[i].V / maxAbsV) * (plotH / 2));
        }
        ctx.lineTo(this.metersToPx(points[points.length - 1].x, beamW), zeroY);
        ctx.closePath();
        ctx.fill();
        ctx.restore();

        // 2. Negative Region Fill (Single Batched Draw Call with Clip to lower half)
        ctx.save();
        ctx.beginPath();
        ctx.rect(this.padding.left, zeroY, beamW, plotH / 2);
        ctx.clip();

        ctx.fillStyle = negGrad;
        ctx.beginPath();
        ctx.moveTo(this.metersToPx(points[0].x, beamW), zeroY);
        for (let i = 0; i < points.length; i++) {
          ctx.lineTo(this.metersToPx(points[i].x, beamW), zeroY - (points[i].V / maxAbsV) * (plotH / 2));
        }
        ctx.lineTo(this.metersToPx(points[points.length - 1].x, beamW), zeroY);
        ctx.closePath();
        ctx.fill();
        ctx.restore();

        // Main line plot of V(x)
        ctx.strokeStyle = colors.sfdLine;
        ctx.beginPath();
        for (let i = 0; i < points.length; i++) {
          const px = this.metersToPx(points[i].x, beamW);
          const py = zeroY - (points[i].V / maxAbsV) * (plotH / 2);
          if (i === 0) ctx.moveTo(px, py);
          else ctx.lineTo(px, py);
        }
        ctx.stroke();

        // Highlight Zero Shear Points
        for (const zp of this.solution.diagnostics.zeroShearPoints) {
          const zx = this.metersToPx(zp.x, beamW);
          this.drawMarker(ctx, zx, zeroY, '#f59e0b', 'V = 0', `${zp.x}m`, colors);
```

```
      }

      ctx.restore();

      // Synchronized Crosshair
      if (this.hoverX !== null) {
        const hx = this.metersToPx(this.hoverX, beamW);
        this.drawCrosshairLine(ctx, hx, 0, height, colors);

        // Active value dot
        const val = this.engine.evaluateAt(this.hoverX, this.solution.reactions);
        const hy = zeroY - (val.V / maxAbsV) * (plotH / 2);
        this.drawCrosshairDot(ctx, hx, hy, colors.sfdLine, colors);
      }
    }

    /**
     * 3. BENDING MOMENT DIAGRAM (BMD) RENDERING
     */
    renderBMD() {
      const ctx = this.bmdCtx;
      const { width, height } = this.resizeCanvas(this.bmdCanvas, ctx);
      const colors = this.getColors();
      const beamW = width - this.padding.left - this.padding.right;
      const plotH = height - this.padding.top - this.padding.bottom;

      ctx.clearRect(0, 0, width, height);

      const points = this.solution.points;
      if (!points || points.length === 0) return;

      // Determine scale: max absolute M
      const maxAbsM = Math.max(1, this.solution.diagnostics.maxAbsM * 1.25);
      const zeroY = this.padding.top + plotH / 2;

      // Draw Grid and Axis
      this.drawDiagramGrid(ctx, width, height, beamW, plotH, zeroY, maxAbsM, 'kN·m', colors);

      ctx.save();
      ctx.lineWidth = 2.2;

      // Gradients
      const sagGrad = ctx.createLinearGradient(0, this.padding.top, 0, zeroY);
      sagGrad.addColorStop(0, colors.bmdFillSag);
      sagGrad.addColorStop(1, 'rgba(16, 185, 129, 0.02)');

      const hogGrad = ctx.createLinearGradient(0, zeroY, 0, height - this.padding.bottom);
      hogGrad.addColorStop(0, 'rgba(168, 85, 247, 0.02)');
      hogGrad.addColorStop(1, colors.bmdFillHog);

      // 1. Sagging Region Fill (Batched single draw call with clip to upper half)
      ctx.save();
      ctx.beginPath();
      ctx.rect(this.padding.left, this.padding.top, beamW, plotH / 2);
      ctx.clip();

      ctx.fillStyle = sagGrad;
      ctx.beginPath();
      ctx.moveTo(this.metersToPx(points[0].x, beamW), zeroY);
      for (let i = 0; i < points.length; i++) {
        ctx.lineTo(this.metersToPx(points[i].x, beamW), zeroY - (points[i].M / maxAbsM) * (plotH / 2));
      }
      ctx.lineTo(this.metersToPx(points[points.length - 1].x, beamW), zeroY);
      ctx.closePath();
      ctx.fill();
      ctx.restore();

      // 2. Hogging Region Fill (Batched single draw call with clip to lower half)
      ctx.save();
      ctx.beginPath();
```

```
      ctx.rect(this.padding.left, zeroY, beamW, plotH / 2);
      ctx.clip();

      ctx.fillStyle = hogGrad;
      ctx.beginPath();
      ctx.moveTo(this.metersToPx(points[0].x, beamW), zeroY);
      for (let i = 0; i < points.length; i++) {
        ctx.lineTo(this.metersToPx(points[i].x, beamW), zeroY - (points[i].M / maxAbsM) * (plotH / 2));
      }
      ctx.lineTo(this.metersToPx(points[points.length - 1].x, beamW), zeroY);
      ctx.closePath();
      ctx.fill();
      ctx.restore();

      // BMD Solid Curve
      ctx.strokeStyle = colors.bmdLine;
      ctx.beginPath();
      for (let i = 0; i < points.length; i++) {
        const px = this.metersToPx(points[i].x, beamW);
        const py = zeroY - (points[i].M / maxAbsM) * (plotH / 2);
        if (i === 0) ctx.moveTo(px, py);
        else ctx.lineTo(px, py);
      }
      ctx.stroke();

      // Mark Peak Sagging Moment (Max M)
      const d = this.solution.diagnostics;
      if (d.maxM !== -Infinity && Math.abs(d.maxM) > 0.01) {
        const mx = this.metersToPx(d.maxMPos, beamW);
        const my = zeroY - (d.maxM / maxAbsM) * (plotH / 2);
        this.drawMomentBadge(ctx, mx, my, `M_max = ${d.maxM} kN·m`, colors.reaction, colors);
      }

      // Mark Peak Hogging Moment (Min M)
      if (d.minM !== Infinity && Math.abs(d.minM) > 0.01 && d.minM < 0) {
        const mx = this.metersToPx(d.minMPos, beamW);
        const my = zeroY - (d.minM / maxAbsM) * (plotH / 2);
        this.drawMomentBadge(ctx, mx, my, `M_min = ${d.minM} kN·m`, colors.moment, colors);
      }

      // Mark Points of Contraflexure (M = 0)
      for (const cp of d.contraflexurePoints) {
        const cx = this.metersToPx(cp.x, beamW);
        this.drawDiamondMarker(ctx, cx, zeroY, '#38bdf8', 'Inflection Point', `${cp.x}m`, colors);
      }

      ctx.restore();

      // Synchronized Crosshair
      if (this.hoverX !== null) {
        const hx = this.metersToPx(this.hoverX, beamW);
        this.drawCrosshairLine(ctx, hx, 0, height, colors);

        const val = this.engine.evaluateAt(this.hoverX, this.solution.reactions);
        const hy = zeroY - (val.M / maxAbsM) * (plotH / 2);
        this.drawCrosshairDot(ctx, hx, hy, colors.bmdLine, colors);
      }
    }

    drawDiagramGrid(ctx, width, height, beamW, plotH, zeroY, maxVal, unit, colors) {
      ctx.save();

      // Horizontal zero axis
      ctx.strokeStyle = colors.axis;
      ctx.lineWidth = 1.5;
      ctx.setLineDash([4, 4]);
      ctx.beginPath();
      ctx.moveTo(this.padding.left, zeroY);
      ctx.lineTo(width - this.padding.right, zeroY);
      ctx.stroke();
```

```
      ctx.setLineDash([]);

      // Zero label on Y axis
      ctx.fillStyle = colors.textMuted;
      ctx.font = '10px "JetBrains Mono", monospace';
      ctx.textAlign = 'right';
      ctx.fillText(`0.0 ${unit}`, this.padding.left - 8, zeroY + 3);

      // +Max and -Max grid lines
      const yTop = this.padding.top;
      const yBottom = height - this.padding.bottom;

      ctx.strokeStyle = colors.grid;
      ctx.lineWidth = 1;

      ctx.beginPath();
      ctx.moveTo(this.padding.left, yTop);
      ctx.lineTo(width - this.padding.right, yTop);
      ctx.moveTo(this.padding.left, yBottom);
      ctx.lineTo(width - this.padding.right, yBottom);
      ctx.stroke();

      ctx.fillText(`+${maxVal.toFixed(1)} ${unit}`, this.padding.left - 8, yTop + 4);
      ctx.fillText(`-${maxVal.toFixed(1)} ${unit}`, this.padding.left - 8, yBottom + 4);

      // Vertical grid ticks
      const numTicks = Math.min(10, Math.ceil(this.engine.length));
      for (let i = 0; i <= numTicks; i++) {
        const xVal = (i / numTicks) * this.engine.length;
        const px = this.metersToPx(xVal, beamW);

        ctx.beginPath();
        ctx.moveTo(px, this.padding.top);
        ctx.lineTo(px, height - this.padding.bottom);
        ctx.stroke();
      }

      ctx.restore();
    }

    drawMarker(ctx, x, y, color, title, subtitle, colors) {
      ctx.save();
      // Halo
      ctx.fillStyle = color;
      ctx.shadowColor = color;
      ctx.shadowBlur = 10;
      ctx.beginPath();
      ctx.arc(x, y, 4.5, 0, Math.PI * 2);
      ctx.fill();

      ctx.shadowColor = 'transparent';
      ctx.fillStyle = colors.cardBg;
      ctx.beginPath();
      ctx.arc(x, y, 2, 0, Math.PI * 2);
      ctx.fill();

      // Floating tag
      ctx.font = 'bold 9px "JetBrains Mono", monospace';
      ctx.textAlign = 'center';
      ctx.fillStyle = color;
      ctx.fillText(`${title} (${subtitle})`, x, y - 10);
      ctx.restore();
    }

    drawDiamondMarker(ctx, x, y, color, title, subtitle, colors) {
      ctx.save();
      const d = 5;
      ctx.fillStyle = color;
      ctx.shadowColor = color;
      ctx.shadowBlur = 8;
```

```
904
905       ctx.beginPath();
906       ctx.moveTo(x, y - d);
907       ctx.lineTo(x + d, y);
908       ctx.lineTo(x, y + d);
909       ctx.lineTo(x - d, y);
910       ctx.closePath();
911       ctx.fill();
912
913       ctx.shadowColor = 'transparent';
914       ctx.font = 'bold 9px "JetBrains Mono", monospace';
915       ctx.textAlign = 'center';
916       ctx.fillStyle = color;
917       ctx.fillText(`Inflection (${subtitle})`, x, y + 14);
918       ctx.restore();
919     }
920
921     drawMomentBadge(ctx, x, y, text, color, colors) {
922       ctx.save();
923       ctx.font = 'bold 10px "JetBrains Mono", monospace';
924       const tw = ctx.measureText(text).width + 12;
925       const th = 18;
926       const isUpper = y < (this.padding.top + 50);
927       const badgeY = isUpper ? y + 8 : y - th - 8;
928
929       ctx.fillStyle = colors.cardBg;
930       ctx.strokeStyle = color;
931       ctx.lineWidth = 1.5;
932       ctx.shadowColor = 'rgba(0, 0, 0, 0.4)';
933       ctx.shadowBlur = 6;
934
935       ctx.beginPath();
936       ctx.roundRect?.(x - tw / 2, badgeY, tw, th, 4);
937       ctx.fill();
938       ctx.stroke();
939
940       ctx.shadowColor = 'transparent';
941       ctx.fillStyle = color;
942       ctx.textAlign = 'center';
943       ctx.fillText(text, x, badgeY + 12);
944
945       // Pointer dot
946       ctx.fillStyle = color;
947       ctx.beginPath();
948       ctx.arc(x, y, 3.5, 0, Math.PI * 2);
949       ctx.fill();
950
951       ctx.restore();
952     }
953
954     drawCrosshairLine(ctx, x, y1, y2, colors) {
955       ctx.save();
956       ctx.strokeStyle = colors.crosshair;
957       ctx.lineWidth = 1.2;
958       ctx.setLineDash([3, 3]);
959       ctx.shadowColor = colors.crosshairGlow;
960       ctx.shadowBlur = 6;
961
962       ctx.beginPath();
963       ctx.moveTo(x, y1);
964       ctx.lineTo(x, y2);
965       ctx.stroke();
966       ctx.restore();
967     }
968
969     drawCrosshairDot(ctx, x, y, color, colors) {
970       ctx.save();
971       ctx.fillStyle = color;
972       ctx.shadowColor = color;
973       ctx.shadowBlur = 10;
```

```
      ctx.beginPath();
      ctx.arc(x, y, 5, 0, Math.PI * 2);
      ctx.fill();

      ctx.fillStyle = colors.cardBg;
      ctx.beginPath();
      ctx.arc(x, y, 2.5, 0, Math.PI * 2);
      ctx.fill();
      ctx.restore();
    }
  }

```

</> 3. Engineering Loading Presets (presets.js)

143 lines • JAVASCRIPT

Preset library of 7 classic civil and mechanical engineering beam load configurations.

```
/**
 * Standard Engineering Beam Presets
 */

export const BEAM_PRESETS = [
  {
    id: 'double-overhang-udl-pl',
    title: 'Standard Double Overhang (UDL + 3 Point Loads)',
    category: 'Overhanging',
    description: '12m beam with 2m left overhang and 2m right overhang, midspan UDL of 15 kN/m, and point loads at
    config: {
      length: 12,
      supportType: 'two-supports',
      supportA: { x: 2, type: 'pinned' },
      supportB: { x: 10, type: 'roller' },
      pointLoads: [
        { x: 0, magnitude: 20, label: 'Tip Load A' },
        { x: 6, magnitude: 40, label: 'Midspan Load' },
        { x: 12, magnitude: 25, label: 'Tip Load B' }
      ],
      udlLoads: [
        { x1: 2, x2: 10, magnitude: 15, label: 'Main Span UDL' }
      ],
      moments: []
    }
  },
  {
    id: 'single-overhang-cantilever-tip',
    title: 'Single Overhang Cantilever Tip Load',
    category: 'Overhanging',
    description: '10m beam with supports at 0m and 7m, carrying a 50 kN tip load on the 3m overhang with a 10 kN/m
    config: {
      length: 10,
      supportType: 'two-supports',
      supportA: { x: 0, type: 'pinned' },
      supportB: { x: 7, type: 'roller' },
      pointLoads: [
        { x: 10, magnitude: 50, label: 'Overhang Tip' }
      ],
      udlLoads: [
        { x1: 0, x2: 7, magnitude: 12, label: 'Interior UDL' }
      ],
      moments: []
    }
  },
  {
    id: 'simply-supported-center-load',
    title: 'Simply Supported Center Point Load',
    category: 'Simply Supported',
    description: 'Classic textbook simply supported 8m beam with a 60 kN concentrated load at the exact center (x
    config: {
      length: 8,
      supportType: 'two-supports',
      supportA: { x: 0, type: 'pinned' },
      supportB: { x: 8, type: 'roller' },
      pointLoads: [
        { x: 4, magnitude: 60, label: 'Center Load P' }
      ],
      udlLoads: [],
      moments: []
    }
  },
  {
    id: 'simply-supported-full-udl',
```

```
      title: 'Simply Supported Full Uniform Load (wL²/8)',
      category: 'Simply Supported',
      description: '10m simply supported beam under continuous 20 kN/m distributed load demonstrating parabolic bend
      config: {
        length: 10,
        supportType: 'two-supports',
        supportA: { x: 0, type: 'pinned' },
        supportB: { x: 10, type: 'roller' },
        pointLoads: [],
        udlLoads: [
          { x1: 0, x2: 10, magnitude: 20, label: 'Full Span UDL' }
        ],
        moments: []
      }
    },
    {
      id: 'symmetric-train-loading',
      title: 'Symmetric Two-Axle Wheel / Crane Loading',
      category: 'Transportation',
      description: '14m bridge girder subjected to two 45 kN wheel axle loads spaced 3m apart traversing near the ce
      config: {
        length: 14,
        supportType: 'two-supports',
        supportA: { x: 1, type: 'pinned' },
        supportB: { x: 13, type: 'roller' },
        pointLoads: [
          { x: 5.5, magnitude: 45, label: 'Axle 1' },
          { x: 8.5, magnitude: 45, label: 'Axle 2' }
        ],
        udlLoads: [
          { x1: 0, x2: 14, magnitude: 5, label: 'Beam Self-Weight' }
        ],
        moments: []
      }
    },
    {
      id: 'overhang-couple-moment',
      title: 'Overhanging Beam with Couple Moment & UDL',
      category: 'Overhanging',
      description: '9m beam with an applied 35 kN·m concentrated moment inducing pure rotational step change in the
      config: {
        length: 9,
        supportType: 'two-supports',
        supportA: { x: 1.5, type: 'pinned' },
        supportB: { x: 7.5, type: 'roller' },
        pointLoads: [
          { x: 0, magnitude: 15, label: 'Left Overhang Load' }
        ],
        udlLoads: [
          { x1: 1.5, x2: 7.5, magnitude: 10, label: 'Main Span UDL' }
        ],
        moments: [
          { x: 4.5, magnitude: 40, label: 'Concentrated Moment' }
        ]
      }
    },
    {
      id: 'pure-cantilever-tip-load',
      title: 'Pure Cantilever with UDL and Tip Moment',
      category: 'Cantilever',
      description: '6m fixed cantilever clamped at x = 0m, with 12 kN/m uniform load and 25 kN·m tip moment at free
      config: {
        length: 6,
        supportType: 'cantilever-left',
        supportA: { x: 0, type: 'pinned' },
        supportB: { x: 6, type: 'roller' },
        pointLoads: [
          { x: 6, magnitude: 30, label: 'Tip Point Load' }
        ],
        udlLoads: [
```

```
          { x1: 0, x2: 4, magnitude: 12, label: 'Distributed Load' }
        ],
        moments: [
          { x: 6, magnitude: 25, label: 'Tip Couple' }
        ]
      }
    }
  ];

```

</> 4. Main Application Controller (app.js)

861 lines • JAVASCRIPT

Reactive state management, two-way input synchronization, live diagnostics, dynamic load list handling, and modal controllers.

```
/**
 * Main Application Controller for Structural Beam Analysis Dashboard
 */

import { BeamEngine } from './beam-engine.js';
import { BeamRenderer } from './beam-renderer.js';
import { BEAM_PRESETS } from './presets.js';

class BeamApp {
  constructor() {
    this.engine = new BeamEngine({
      length: 12,
      supportType: 'two-supports',
      supportA: { x: 2, type: 'pinned' },
      supportB: { x: 10, type: 'roller' },
      pointLoads: [
        { id: 'pl_1', x: 0, magnitude: 20, label: 'Tip Load A' },
        { id: 'pl_2', x: 6, magnitude: 40, label: 'Midspan Load' },
        { id: 'pl_3', x: 12, magnitude: 25, label: 'Tip Load B' }
      ],
      udlLoads: [
        { id: 'udl_1', x1: 2, x2: 10, magnitude: 15, label: 'Main Span UDL' }
      ],
      moments: [],
      sectionModulus: 500,
      allowableStress: 250,
      signConvention: 'sagging-positive'
    });

    this.activeTab = 'all';
    this.isDarkMode = true;
    this.currentSolution = null;

    this.initDOM();
    this.initRenderer();
    this.initEventListeners();
    this.loadPresetsList();
    this.update();
  }

  initDOM() {
    // Elements Cache
    this.dom = {
      // Inputs
      sliderLength: document.getElementById('slider-length'),
      numLength: document.getElementById('num-length'),
      valLength: document.getElementById('val-length'),
      selectSupportSystem: document.getElementById('select-support-system'),
      twoSupportsControls: document.getElementById('two-supports-controls'),
      sliderSupportA: document.getElementById('slider-support-a'),
      numSupportA: document.getElementById('num-support-a'),
      valSupportA: document.getElementById('val-support-a'),
      sliderSupportB: document.getElementById('slider-support-b'),
      numSupportB: document.getElementById('num-support-b'),
      valSupportB: document.getElementById('val-support-b'),
      overhangLeftPill: document.getElementById('overhang-left-pill'),
      overhangRightPill: document.getElementById('overhang-right-pill'),
      spanInfoBadge: document.getElementById('span-info-badge'),

      // Stress & Section
      selectProfileType: document.getElementById('select-profile-type'),
      rectInputs: document.getElementById('rect-inputs'),
      rectB: document.getElementById('rect-b'),
```

```
        rectH: document.getElementById('rect-h'),
        sliderSectionModulus: document.getElementById('slider-section-modulus'),
        numSectionModulus: document.getElementById('num-section-modulus'),
        valSectionModulus: document.getElementById('val-section-modulus'),
        selectMaterial: document.getElementById('select-material'),
        sliderAllowableStress: document.getElementById('slider-allowable-stress'),
        numAllowableStress: document.getElementById('num-allowable-stress'),
        valAllowableStress: document.getElementById('val-allowable-stress'),
        stressBadge: document.getElementById('stress-badge'),

        // Loads
        loadsContainer: document.getElementById('loads-container'),
        countAll: document.getElementById('count-all'),
        countPoint: document.getElementById('count-point'),
        countUDL: document.getElementById('count-udl'),
        countMoment: document.getElementById('count-moment'),

        // Metrics
        equilibriumBadge: document.getElementById('equilibrium-badge'),
        metricEquilibrium: document.getElementById('metric-equilibrium'),
        metricLoadSum: document.getElementById('metric-load-sum'),
        metricReactions: document.getElementById('metric-reactions'),
        metricReactionsSub: document.getElementById('metric-reactions-sub'),
        metricMaxMoment: document.getElementById('metric-max-moment'),
        metricMomentPos: document.getElementById('metric-moment-pos'),
        metricMaxShear: document.getElementById('metric-max-shear'),
        metricShearPos: document.getElementById('metric-shear-pos'),
        metricStress: document.getElementById('metric-stress'),
        metricSafetyFactor: document.getElementById('metric-safety-factor'),
        schematicBadge: document.getElementById('schematic-badge'),
        zeroShearBadge: document.getElementById('zero-shear-badge'),
        contraflexureBadge: document.getElementById('contraflexure-badge'),

        // Diagnostics Tables & Math
        equilibriumStepsContent: document.getElementById('equilibrium-steps-content'),
        criticalPointsTableBody: document.getElementById('critical-points-table-body'),
        critPointsCount: document.getElementById('crit-points-count'),

        // HUD
        crosshairHud: document.getElementById('crosshair-hud'),
        hudX: document.getElementById('hud-x'),
        hudV: document.getElementById('hud-v'),
        hudM: document.getElementById('hud-m'),
        hudStress: document.getElementById('hud-stress'),
        hudStatus: document.getElementById('hud-status'),

        // Modals
        modalAddLoad: document.getElementById('modal-add-load'),
        modalPresets: document.getElementById('modal-presets'),
        modalLatex: document.getElementById('modal-latex'),

        // Add Load Modal fields
        modalLoadType: document.getElementById('modal-load-type'),
        modalFieldsPoint: document.getElementById('modal-fields-point'),
        modalFieldsUDL: document.getElementById('modal-fields-udl'),
        modalFieldsMoment: document.getElementById('modal-fields-moment'),
        modalPlX: document.getElementById('modal-pl-x'),
        modalPlMag: document.getElementById('modal-pl-mag'),
        modalPlDir: document.getElementById('modal-pl-dir'),
        modalUdlX1: document.getElementById('modal-udl-x1'),
        modalUdlX2: document.getElementById('modal-udl-x2'),
        modalUdlMag: document.getElementById('modal-udl-mag'),
        modalMomentX: document.getElementById('modal-moment-x'),
        modalMomentMag: document.getElementById('modal-moment-mag'),
        modalMomentDir: document.getElementById('modal-moment-dir'),

        // Buttons & Actions
        btnGenerateDiagrams: document.getElementById('btn-generate-diagrams'),
        btnHeaderGenerate: document.getElementById('btn-header-generate'),
        btnCalculateLoads: document.getElementById('btn-calculate-loads'),
```

```
        btnQuickAdd: document.getElementById('btn-quick-add'),
        quickLoadType: document.getElementById('quick-load-type'),
        quickLoadX: document.getElementById('quick-load-x'),
        quickLoadMag: document.getElementById('quick-load-mag'),
        toastNotification: document.getElementById('toast-notification'),
        toastMsg: document.getElementById('toast-msg'),

        btnTheme: document.getElementById('btn-theme'),
        themeIcon: document.getElementById('theme-icon'),
        btnConvention: document.getElementById('btn-convention'),
        btnPresets: document.getElementById('btn-presets'),
        btnLatex: document.getElementById('btn-latex'),
        btnExportCsv: document.getElementById('btn-export-csv'),
        btnExportPrint: document.getElementById('btn-export-print'),
        btnAddLoadModal: document.getElementById('btn-add-load-modal'),
        btnSaveNewLoad: document.getElementById('btn-save-new-load'),
        btnCopyLatex: document.getElementById('btn-copy-latex'),
        presetsListContainer: document.getElementById('presets-list-container')
      };
    }

    initRenderer() {
      const canvases = {
        beam: document.getElementById('canvas-beam'),
        sfd: document.getElementById('canvas-sfd'),
        bmd: document.getElementById('canvas-bmd')
      };

      this.renderer = new BeamRenderer(canvases, { isDarkMode: this.isDarkMode });

      // Synchronized crosshair HUD callback
      this.renderer.onInspect((x, clientX, clientY) => {
        if (x === null) {
          this.dom.crosshairHud.style.display = 'none';
          return;
        }

        this.dom.crosshairHud.style.display = 'flex';
        if (clientX !== undefined && clientY !== undefined) {
          this.dom.crosshairHud.style.left = `${clientX}px`;
          this.dom.crosshairHud.style.top = `${clientY}px`;
        }

        const val = this.engine.evaluateAt(x, this.currentSolution.reactions);
        this.dom.hudX.textContent = `${x.toFixed(2)} m`;
        this.dom.hudV.textContent = `${val.V.toFixed(2)} kN`;
        this.dom.hudM.textContent = `${val.M.toFixed(2)} kN·m`;

        const stress = this.engine.sectionModulus > 0 ? (Math.abs(val.M) * 1000) / this.engine.sectionModulus : 0;
        this.dom.hudStress.textContent = `${stress.toFixed(1)} MPa`;

        // Status text
        let status = 'Span Section';
        if (Math.abs(x - this.engine.supportA.x) < 0.1 && this.engine.supportType === 'two-supports') status = 'Supp
        else if (Math.abs(x - this.engine.supportB.x) < 0.1 && this.engine.supportType === 'two-supports') status =
        else if (x < this.engine.supportA.x && this.engine.supportType === 'two-supports') status = 'Left Overhang';
        else if (x > this.engine.supportB.x && this.engine.supportType === 'two-supports') status = 'Right Overhang'
        else if (this.engine.pointLoads.some(p => Math.abs(p.x - x) < 0.1)) status = 'Point Load Section';
        else if (this.engine.udlLoads.some(u => x >= u.x1 && x <= u.x2)) status = 'Under Distributed Load';

        this.dom.hudStatus.textContent = status;
      });

      window.addEventListener('resize', () => {
        this.render();
      });
    }

    initEventListeners() {
      // 1. Beam Length
```

```
      const updateLength = (val) => {
        const L = Math.max(1, Number(val));
        this.engine.setBeamLength(L);
        this.dom.valLength.textContent = `${L.toFixed(2)} m`;
        this.dom.sliderLength.value = L;
        this.dom.numLength.value = L;

        // Update max slider limits for supports
        this.dom.sliderSupportA.max = L;
        this.dom.sliderSupportB.max = L;
        this.dom.numSupportA.max = L;
        this.dom.numSupportB.max = L;

        // Clamp existing supports inside length
        if (this.engine.supportA.x > L) this.engine.supportA.x = Math.max(0, L - 1);
        if (this.engine.supportB.x > L) this.engine.supportB.x = L;

        this.syncSupportInputs();
        this.update();
      };

      this.dom.sliderLength.addEventListener('input', (e) => updateLength(e.target.value));
      this.dom.numLength.addEventListener('change', (e) => updateLength(e.target.value));

      // 2. Support Type Selector
      this.dom.selectSupportSystem.addEventListener('change', (e) => {
        const type = e.target.value;
        this.engine.supportType = type;
        if (type === 'two-supports') {
          this.dom.twoSupportsControls.style.display = 'block';
        } else {
          this.dom.twoSupportsControls.style.display = 'none';
        }
        this.update();
      });

      // 3. Support A Position
      const updateSupportA = (val) => {
        const posA = Math.max(0, Math.min(this.engine.length, Number(val)));
        this.engine.supportA.x = posA;
        this.dom.valSupportA.textContent = `${posA.toFixed(2)} m`;
        this.dom.sliderSupportA.value = posA;
        this.dom.numSupportA.value = posA;
        this.update();
      };

      this.dom.sliderSupportA.addEventListener('input', (e) => updateSupportA(e.target.value));
      this.dom.numSupportA.addEventListener('change', (e) => updateSupportA(e.target.value));

      // 4. Support B Position
      const updateSupportB = (val) => {
        const posB = Math.max(0, Math.min(this.engine.length, Number(val)));
        this.engine.supportB.x = posB;
        this.dom.valSupportB.textContent = `${posB.toFixed(2)} m`;
        this.dom.sliderSupportB.value = posB;
        this.dom.numSupportB.value = posB;
        this.update();
      };

      this.dom.sliderSupportB.addEventListener('input', (e) => updateSupportB(e.target.value));
      this.dom.numSupportB.addEventListener('change', (e) => updateSupportB(e.target.value));

      // 5. Section Modulus Z
      const updateZ = (val) => {
        const z = Math.max(1, Number(val));
        this.engine.sectionModulus = z;
        this.dom.valSectionModulus.textContent = `${z} cm³`;
        this.dom.sliderSectionModulus.value = z;
        this.dom.numSectionModulus.value = z;
        this.update();
```

```
      };

      this.dom.sliderSectionModulus.addEventListener('input', (e) => updateZ(e.target.value));
      this.dom.numSectionModulus.addEventListener('change', (e) => updateZ(e.target.value));

      // 6. Section Profile Type Calculator
      this.dom.selectProfileType.addEventListener('change', (e) => {
        const type = e.target.value;
        if (type === 'rect') {
          this.dom.rectInputs.style.display = 'block';
          this.calcRectZ();
        } else if (type === 'i-beam') {
          this.dom.rectInputs.style.display = 'none';
          updateZ(557); // IPE 300 W_el = 557 cm^3
        } else if (type === 'circle') {
          this.dom.rectInputs.style.display = 'none';
          // Solid round d = 200mm => Z = pi*d^3/32 = 785 cm^3
          updateZ(785);
        } else {
          this.dom.rectInputs.style.display = 'none';
        }
      });

      this.dom.rectB.addEventListener('input', () => this.calcRectZ());
      this.dom.rectH.addEventListener('input', () => this.calcRectZ());

      // 7. Allowable Stress & Material
      const updateAllowableStress = (val) => {
        const s = Math.max(1, Number(val));
        this.engine.allowableStress = s;
        this.dom.valAllowableStress.textContent = `${s} MPa`;
        this.dom.sliderAllowableStress.value = s;
        this.dom.numAllowableStress.value = s;
        this.update();
      };

      this.dom.sliderAllowableStress.addEventListener('input', (e) => updateAllowableStress(e.target.value));
      this.dom.numAllowableStress.addEventListener('change', (e) => updateAllowableStress(e.target.value));

      this.dom.selectMaterial.addEventListener('change', (e) => {
        if (e.target.value !== 'custom') {
          updateAllowableStress(Number(e.target.value));
        }
      });

      // 8. Sign Convention Toggle
      this.dom.btnConvention.addEventListener('click', () => {
        if (this.engine.signConvention === 'sagging-positive') {
          this.engine.signConvention = 'hogging-positive';
          this.dom.btnConvention.innerHTML = `<i data-lucide="sliders"></i> Hogging +`;
        } else {
          this.engine.signConvention = 'sagging-positive';
          this.dom.btnConvention.innerHTML = `<i data-lucide="sliders"></i> Sagging +`;
        }
        lucide?.createIcons();
        this.update();
      });

      // 9. Load Filter Tabs
      document.querySelectorAll('.tab-btn').forEach(btn => {
        btn.addEventListener('click', () => {
          document.querySelectorAll('.tab-btn').forEach(b => b.classList.remove('active'));
          btn.classList.add('active');
          this.activeTab = btn.getAttribute('data-tab');
          this.renderLoadList();
        });
      });

      // 10. Modals Management
      this.dom.btnAddLoadModal.addEventListener('click', () => {
```

```
        this.openModal(this.dom.modalAddLoad);
      });

      this.dom.btnPresets.addEventListener('click', () => {
        this.openModal(this.dom.modalPresets);
      });

      this.dom.btnLatex.addEventListener('click', () => {
        this.showLatexModal();
      });

      // Close buttons on all modals
      document.querySelectorAll('.btn-modal-close').forEach(btn => {
        btn.addEventListener('click', () => {
          document.querySelectorAll('.modal-overlay').forEach(m => m.classList.remove('active'));
        });
      });

      // Add Load Type Switcher in modal
      this.dom.modalLoadType.addEventListener('change', (e) => {
        const type = e.target.value;
        this.dom.modalFieldsPoint.style.display = type === 'point' ? 'block' : 'none';
        this.dom.modalFieldsUDL.style.display = type === 'udl' ? 'block' : 'none';
        this.dom.modalFieldsMoment.style.display = type === 'moment' ? 'block' : 'none';
      });

      // Save New Load
      this.dom.btnSaveNewLoad.addEventListener('click', () => {
        this.saveLoadFromModal();
      });

      // Copy LaTeX
      this.dom.btnCopyLatex.addEventListener('click', () => {
        if (this.currentLatex) {
          const text = `% Structural Beam Analysis LaTeX Expressions\n% Reactions:\n${this.currentLatex.reactionsLat
          navigator.clipboard.writeText(text).then(() => {
            this.dom.btnCopyLatex.innerHTML = `<i data-lucide="check"></i> Copied!`;
            lucide?.createIcons();
            setTimeout(() => {
              this.dom.btnCopyLatex.innerHTML = `<i data-lucide="copy"></i> Copy LaTeX`;
              lucide?.createIcons();
            }, 2000);
          });
        }
      });

      // Export CSV
      this.dom.btnExportCsv.addEventListener('click', () => {
        this.exportCSV();
      });

      // Export PDF / Print
      this.dom.btnExportPrint.addEventListener('click', () => {
        document.getElementById('print-timestamp').textContent = `Generated on: ${new Date().toLocaleString()} | Bea
        window.print();
      });

      // Theme Toggle
      this.dom.btnTheme.addEventListener('click', () => {
        this.isDarkMode = !this.isDarkMode;
        document.documentElement.setAttribute('data-theme', this.isDarkMode ? 'dark' : 'light');
        this.dom.themeIcon.setAttribute('data-lucide', this.isDarkMode ? 'moon' : 'sun');
        this.renderer.setTheme(this.isDarkMode);
        lucide?.createIcons();
        this.render();
      });

      // 11. Explicit "Generate SFD & BMD" Buttons
      // Main Hero CTA button
      this.dom.btnGenerateDiagrams?.addEventListener('click', () => {
```

```
        this.generateSFDandBMD('hero-button');
      });

      // Header CTA button
      this.dom.btnHeaderGenerate?.addEventListener('click', () => {
        this.generateSFDandBMD('header-button');
      });

      // Card CTA button
      this.dom.btnCalculateLoads?.addEventListener('click', () => {
        this.generateSFDandBMD('card-button');
      });

      // Quick Add Button
      this.dom.btnQuickAdd?.addEventListener('click', () => {
        this.addQuickLoad();
      });

      // Keyboard Shortcut: Pressing 'Enter' triggers generation
      window.addEventListener('keydown', (e) => {
        if (e.key === 'Enter' && !document.querySelector('.modal-overlay.active')) {
          this.generateSFDandBMD('keyboard-enter');
        }
      });
    }

    showToast(msg = 'SFD & BMD Generated Successfully') {
      if (!this.dom.toastNotification) return;
      this.dom.toastMsg.textContent = msg;
      this.dom.toastNotification.classList.add('show');
      clearTimeout(this.toastTimer);
      this.toastTimer = setTimeout(() => {
        this.dom.toastNotification.classList.remove('show');
      }, 2500);
    }

    generateSFDandBMD(source = 'button') {
      // 1. Solve Section Mechanics & Equilibrium
      this.update();

      // 2. Force Canvas re-rendering across all 3 viewports
      this.render();

      // 3. Trigger User Feedback Animation & Toast
      const pointsCount = this.currentSolution?.points?.length || 2000;
      this.showToast(`⚡ SFD & BMD Generated (${pointsCount.toLocaleString()} points calculated)`);

      // Subtle button click feedback animation
      const btn = this.dom.btnGenerateDiagrams;
      if (btn) {
        btn.style.transform = 'scale(0.97)';
        setTimeout(() => { btn.style.transform = ''; }, 150);
      }
    }

    addQuickLoad() {
      const type = this.dom.quickLoadType.value;
      const x = Math.max(0, Math.min(this.engine.length, Number(this.dom.quickLoadX.value) || 0));
      const rawMag = Number(this.dom.quickLoadMag.value) || 20;

      if (type === 'point') {
        this.engine.addPointLoad({
          x,
          magnitude: Math.abs(rawMag), // default gravity load
          label: `P${this.engine.pointLoads.length + 1}`
        });
      } else if (type === 'udl') {
        const x2 = Math.min(this.engine.length, x + Math.max(1, this.engine.length * 0.25));
        this.engine.addUDL({
          x1: x,
```

```
          x2: Number(x2.toFixed(2)),
          magnitude: Math.abs(rawMag),
          label: `w${this.engine.udlLoads.length + 1}`
        });
      } else if (type === 'moment') {
        this.engine.addMoment({
          x,
          magnitude: rawMag,
          label: `M${this.engine.moments.length + 1}`
        });
      }

      this.generateSFDandBMD('quick-add');
    }

    calcRectZ() {
      const b = Number(this.dom.rectB.value) || 150; // mm
      const h = Number(this.dom.rectH.value) || 300; // mm
      // Z = b * h^2 / 6 in mm^3, convert to cm^3 (/ 1000)
      const z_cm3 = Math.round((b * h * h) / (6 * 1000));
      this.engine.sectionModulus = z_cm3;
      this.dom.valSectionModulus.textContent = `${z_cm3} cm³`;
      this.dom.sliderSectionModulus.value = z_cm3;
      this.dom.numSectionModulus.value = z_cm3;
      this.update();
    }

    openModal(modal) {
      modal.classList.add('active');
      lucide?.createIcons();
    }

    saveLoadFromModal() {
      const type = this.dom.modalLoadType.value;
      const L = this.engine.length;

      if (type === 'point') {
        const x = Math.max(0, Math.min(L, Number(this.dom.modalPlX.value)));
        const rawMag = Number(this.dom.modalPlMag.value);
        const isDown = this.dom.modalPlDir.value === 'down';
        const magnitude = isDown ? Math.abs(rawMag) : -Math.abs(rawMag);

        this.engine.addPointLoad({ x, magnitude, label: `P${this.engine.pointLoads.length + 1}` });
      } else if (type === 'udl') {
        const x1 = Math.max(0, Math.min(L, Number(this.dom.modalUdlX1.value)));
        const x2 = Math.max(0, Math.min(L, Number(this.dom.modalUdlX2.value)));
        const magnitude = Math.abs(Number(this.dom.modalUdlMag.value));

        this.engine.addUDL({ x1, x2, magnitude, label: `w${this.engine.udlLoads.length + 1}` });
      } else if (type === 'moment') {
        const x = Math.max(0, Math.min(L, Number(this.dom.modalMomentX.value)));
        const rawMag = Number(this.dom.modalMomentMag.value);
        const isCW = this.dom.modalMomentDir.value === 'cw';
        const magnitude = isCW ? Math.abs(rawMag) : -Math.abs(rawMag);

        this.engine.addMoment({ x, magnitude, label: `M${this.engine.moments.length + 1}` });
      }

      this.dom.modalAddLoad.classList.remove('active');
      this.generateSFDandBMD('modal-add');
    }

    syncSupportInputs() {
      this.dom.sliderSupportA.value = this.engine.supportA.x;
      this.dom.numSupportA.value = this.engine.supportA.x;
      this.dom.valSupportA.textContent = `${this.engine.supportA.x.toFixed(2)} m`;

      this.dom.sliderSupportB.value = this.engine.supportB.x;
      this.dom.numSupportB.value = this.engine.supportB.x;
      this.dom.valSupportB.textContent = `${this.engine.supportB.x.toFixed(2)} m`;
```

```
554     }
555
556     loadPresetsList() {
557       this.dom.presetsListContainer.innerHTML = '';
558       for (const preset of BEAM_PRESETS) {
559         const card = document.createElement('div');
560         card.className = 'preset-card';
561         card.innerHTML = `
562           <div class="preset-title">
563             <span>${preset.title}</span>
564             <span class="badge badge-info" style="font-size: 0.65rem;">${preset.category}</span>
565           </div>
566           <p class="preset-desc">${preset.description}</p>
567         `;
568
569         card.addEventListener('click', () => {
570           this.applyPreset(preset);
571           this.dom.modalPresets.classList.remove('active');
572         });
573
574         this.dom.presetsListContainer.appendChild(card);
575       }
576     }
577
578     applyPreset(preset) {
579       const c = preset.config;
580       this.engine.setBeamLength(c.length);
581       this.engine.supportType = c.supportType;
582       this.engine.supportA = { ...c.supportA };
583       this.engine.supportB = { ...c.supportB };
584       this.engine.clearLoads();
585
586       for (const pl of c.pointLoads) this.engine.addPointLoad(pl);
587       for (const udl of c.udlLoads) this.engine.addUDL(udl);
588       for (const m of c.moments) this.engine.addMoment(m);
589
590       // Sync UI controls
591       this.dom.sliderLength.value = c.length;
592       this.dom.numLength.value = c.length;
593       this.dom.valLength.textContent = `${c.length.toFixed(2)} m`;
594
595       this.dom.selectSupportSystem.value = c.supportType;
596       if (c.supportType === 'two-supports') {
597         this.dom.twoSupportsControls.style.display = 'block';
598       } else {
599         this.dom.twoSupportsControls.style.display = 'none';
600       }
601
602       this.syncSupportInputs();
603       this.update();
604     }
605
606     renderLoadList() {
607       const container = this.dom.loadsContainer;
608       container.innerHTML = '';
609
610       const items = [];
611
612       if (this.activeTab === 'all' || this.activeTab === 'point') {
613         this.engine.pointLoads.forEach(pl => items.push({ type: 'point', load: pl }));
614       }
615       if (this.activeTab === 'all' || this.activeTab === 'udl') {
616         this.engine.udlLoads.forEach(udl => items.push({ type: 'udl', load: udl }));
617       }
618       if (this.activeTab === 'all' || this.activeTab === 'moment') {
619         this.engine.moments.forEach(m => items.push({ type: 'moment', load: m }));
620       }
621
622       if (items.length === 0) {
623         container.innerHTML = `<div style="text-align: center; color: var(--text-dim); padding: 1.5rem; font-size: 0
```

```
        return;
      }

      items.forEach(({ type, load }) => {
        const el = document.createElement('div');
        el.className = 'load-item';

        let color = 'var(--accent-amber)';
        let desc = '';

        if (type === 'point') {
          color = 'var(--accent-amber)';
          const dir = load.magnitude >= 0 ? '↓ Down' : '↑ Up';
          desc = `<span>${load.label} @ <strong>${load.x.toFixed(2)}m</strong></span> &bull; <span class="load-val">
        } else if (type === 'udl') {
          color = 'var(--accent-cyan)';
          desc = `<span>${load.label} [<strong>${load.x1.toFixed(2)} - ${load.x2.toFixed(2)}m</strong>]</span> &bull
        } else if (type === 'moment') {
          color = 'var(--accent-purple)';
          const rot = load.magnitude >= 0 ? '↻ CW' : '↺ CCW';
          desc = `<span>${load.label} @ <strong>${load.x.toFixed(2)}m</strong></span> &bull; <span class="load-val">
        }

        el.innerHTML = `
          <div class="load-meta">
            <div class="load-color-tag" style="background: ${color};"></div>
            <div>${desc}</div>
          </div>
          <div class="load-actions">
            <button class="btn btn-icon btn-danger btn-delete-load" title="Delete Load" style="padding: 0.25rem 0.4r
              <i data-lucide="trash-2" style="width: 13px; height: 13px;"></i>
            </button>
          </div>
        `;

        el.querySelector('.btn-delete-load').addEventListener('click', () => {
          this.engine.removeLoad(type, load.id);
          this.update();
        });

        container.appendChild(el);
      });

      lucide?.createIcons();
    }

    showLatexModal() {
      const equations = this.engine.generateLatexEquations(this.currentSolution.reactions);
      this.currentLatex = equations;

      const rEl = document.getElementById('latex-reactions-render');
      const sEl = document.getElementById('latex-shear-render');
      const mEl = document.getElementById('latex-moment-render');

      if (window.katex) {
        katex.render(equations.reactionsLatex, rEl, { displayMode: true });
        katex.render(equations.shearLatex, sEl, { displayMode: true });
        katex.render(equations.momentLatex, mEl, { displayMode: true });
      } else {
        rEl.textContent = equations.reactionsLatex;
        sEl.textContent = equations.shearLatex;
        mEl.textContent = equations.momentLatex;
      }

      this.openModal(this.dom.modalLatex);
    }

    exportCSV() {
      const csvContent = this.engine.exportCSV(this.currentSolution);
      const blob = new Blob([csvContent], { type: 'text/csv;charset=utf-8;' });
```

```
    const url = URL.createObjectURL(blob);
    const link = document.createElement('a');
    link.setAttribute('href', url);
    link.setAttribute('download', `beam_analysis_L${this.engine.length}m_${Date.now()}.csv`);
    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
  }

  update() {
    // 1. Solve Section Mechanics
    this.currentSolution = this.engine.solve();
    const sol = this.currentSolution;
    const diag = sol.diagnostics;

    // 2. Update Counts
    this.dom.countAll.textContent = this.engine.pointLoads.length + this.engine.udlLoads.length + this.engine.mome
    this.dom.countPoint.textContent = this.engine.pointLoads.length;
    this.dom.countUDL.textContent = this.engine.udlLoads.length;
    this.dom.countMoment.textContent = this.engine.moments.length;

    // 3. Update Span indicators
    if (this.engine.supportType === 'two-supports') {
      const leftOverhang = Math.max(0, this.engine.supportA.x);
      const mainSpan = Math.abs(this.engine.supportB.x - this.engine.supportA.x);
      const rightOverhang = Math.max(0, this.engine.length - this.engine.supportB.x);

      this.dom.spanInfoBadge.textContent = `Main Span: ${mainSpan.toFixed(2)} m`;
      this.dom.overhangLeftPill.textContent = `Left Overhang: ${leftOverhang.toFixed(2)} m`;
      this.dom.overhangRightPill.textContent = `Right Overhang: ${rightOverhang.toFixed(2)} m`;
    } else {
      this.dom.spanInfoBadge.textContent = `Cantilever: ${this.engine.length.toFixed(2)} m`;
    }

    // 4. Update Header Equilibrium Badge
    if (diag.balanceCheck) {
      this.dom.equilibriumBadge.className = 'badge badge-success';
      this.dom.equilibriumBadge.innerHTML = `<i data-lucide="check-circle-2"></i> ∑Fy = 0.00 kN [PASS]`;
    } else {
      this.dom.equilibriumBadge.className = 'badge badge-danger';
      this.dom.equilibriumBadge.innerHTML = `<i data-lucide="alert-triangle"></i> Balance Error: ${sol.reactions.b
    }

    // 5. Update Metrics Cards
    this.dom.metricEquilibrium.textContent = `Δ = ${sol.reactions.balanceError.toFixed(3)} kN`;
    this.dom.metricLoadSum.textContent = `∑F_down: ${diag.totalLoad.toFixed(2)} kN`;

    if (this.engine.supportType === 'two-supports') {
      this.dom.metricReactions.textContent = `R_A: ${sol.reactions.R1.toFixed(1)} | R_B: ${sol.reactions.R2.toFixe
      this.dom.metricReactionsSub.textContent = `Total Reaction: ${diag.totalReaction.toFixed(2)} kN`;
    } else {
      const r = sol.reactions.R1 || sol.reactions.R2;
      this.dom.metricReactions.textContent = `R_fixed: ${r.toFixed(1)} kN`;
      this.dom.metricReactionsSub.textContent = `M_fixed: ${Math.abs(sol.reactions.M_fixed).toFixed(1)} kN·m`;
    }

    this.dom.metricMaxMoment.textContent = `${diag.maxAbsM.toFixed(1)} kN·m`;
    const isSagging = Math.abs(diag.maxM) >= Math.abs(diag.minM);
    this.dom.metricMomentPos.textContent = `At x = ${isSagging ? diag.maxMPos : diag.minMPos} m (${isSagging ? 'Sa

    this.dom.metricMaxShear.textContent = `${diag.maxAbsV.toFixed(1)} kN`;
    this.dom.metricShearPos.textContent = `At x = ${diag.maxVPos} m`;

    this.dom.metricStress.textContent = `${diag.peakStress} MPa`;
    this.dom.metricSafetyFactor.textContent = `SF: ${diag.safetyFactor} (Allowable: ${this.engine.allowableStress}

    // Stress Badge
    if (diag.stressRatio <= 80) {
      this.dom.stressBadge.className = 'badge badge-success';
      this.dom.stressBadge.textContent = `Safe: ${diag.stressRatio}%`;
```

```
764       } else if (diag.stressRatio <= 100) {
765         this.dom.stressBadge.className = 'badge badge-warning';
766         this.dom.stressBadge.textContent = `Caution: ${diag.stressRatio}%`;
767       } else {
768         this.dom.stressBadge.className = 'badge badge-danger';
769         this.dom.stressBadge.textContent = `Overstressed: ${diag.stressRatio}%`;
770       }
771
772       // Badges on Diagram Headers
773       this.dom.schematicBadge.textContent = `L = ${this.engine.length.toFixed(1)} m`;
774
775       if (diag.zeroShearPoints.length > 0) {
776         const firstZero = diag.zeroShearPoints[0];
777         this.dom.zeroShearBadge.textContent = `V=0 @ x = ${firstZero.x} m (M = ${firstZero.M} kN·m)`;
778       } else {
779         this.dom.zeroShearBadge.textContent = `No interior V=0 crossings`;
780       }
781
782       this.dom.contraflexureBadge.textContent = `${diag.contraflexurePoints.length} Inflection Point${diag.contrafle
783
784       // 6. Update Equilibrium Math Steps Box
785       this.dom.equilibriumStepsContent.textContent = sol.reactions.steps.join('\n');
786
787       // 7. Update Critical Points Table
788       this.renderCriticalPointsTable(diag);
789
790       // 8. Update Dynamic Load Items List
791       this.renderLoadList();
792
793       // 9. Render Canvas Plots
794       this.render();
795
796       lucide?.createIcons();
797     }
798
799     renderCriticalPointsTable(diag) {
800       const tbody = this.dom.criticalPointsTableBody;
801       tbody.innerHTML = '';
802
803       const allCrit = [];
804
805       // Support points
806       if (this.engine.supportType === 'two-supports') {
807         const evalA = this.engine.evaluateAt(this.engine.supportA.x, this.currentSolution.reactions);
808         allCrit.push({ x: this.engine.supportA.x, V: evalA.V, M: evalA.M, type: 'Support A (Pin Reaction)' });
809
810         const evalB = this.engine.evaluateAt(this.engine.supportB.x, this.currentSolution.reactions);
811         allCrit.push({ x: this.engine.supportB.x, V: evalB.V, M: evalB.M, type: 'Support B (Roller Reaction)' });
812       }
813
814       // Zero shear points
815       diag.zeroShearPoints.forEach(zp => allCrit.push({ x: zp.x, V: 0, M: zp.M, type: 'Zero Shear / Local Peak Momen
816
817       // Contraflexure inflection points
818       diag.contraflexurePoints.forEach(cp => allCrit.push({ x: cp.x, V: cp.V, M: 0, type: 'Point of Contraflexure (M
819
820       // Peak Extreme points
821       allCrit.push({ x: diag.maxMPos, V: this.engine.evaluateAt(diag.maxMPos, this.currentSolution.reactions).V, M:
822       if (diag.minM < 0) {
823         allCrit.push({ x: diag.minMPos, V: this.engine.evaluateAt(diag.minMPos, this.currentSolution.reactions).V, M
824       }
825
826       // Sort by x
827       allCrit.sort((a, b) => a.x - b.x);
828
829       // Remove duplicates within 0.05m
830       const filtered = [];
831       for (const pt of allCrit) {
832         if (!filtered.some(p => Math.abs(p.x - pt.x) < 0.02 && p.type === pt.type)) {
833           filtered.push(pt);
```

```
        }
      }

      this.dom.critPointsCount.textContent = `${filtered.length} Points`;

      filtered.forEach(pt => {
        const tr = document.createElement('tr');
        tr.innerHTML = `
          <td style="font-weight: 600; color: var(--accent-cyan);">${pt.x.toFixed(2)}</td>
          <td>${pt.V.toFixed(2)}</td>
          <td>${pt.M.toFixed(2)}</td>
          <td style="color: var(--text-muted); font-size: 0.72rem;">${pt.type}</td>
        `;
        tbody.appendChild(tr);
      });
    }

    render() {
      this.renderer.render(this.engine, this.currentSolution);
    }
  }

  // Start Application on DOM Load
  window.addEventListener('DOMContentLoaded', () => {
    window.app = new BeamApp();
    lucide?.createIcons();
  });
```

</> 5. Styling & CAD Design System (styles.css)

900 lines • CSS

Dark/light mode themes, glassmorphism, responsive grid layout, and print media formatting.

```css
/* ============================================================================
   Structural Beam Analysis Dashboard - Design System & Modern CAD Theme
   Dark Slate (#0f172a), Indigo (#6366f1), Cyan (#06b6d4)
   ============================================================================ */

@import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&family=JetBrains+Mono:wght@40

:root {
  --font-sans: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
  --font-mono: 'JetBrains Mono', monospace;

  /* Dark Theme Palette (Default) */
  --bg-primary: #0a0f1d;
  --bg-surface: #0f172a;
  --bg-card: rgba(30, 41, 59, 0.75);
  --bg-card-hover: rgba(30, 41, 59, 0.95);
  --bg-input: #1e293b;
  --border-subtle: rgba(255, 255, 255, 0.08);
  --border-accent: rgba(99, 102, 241, 0.35);

  --text-main: #f8fafc;
  --text-muted: #94a3b8;
  --text-dim: #64748b;

  --accent-indigo: #6366f1;
  --accent-indigo-hover: #4f46e5;
  --accent-cyan: #06b6d4;
  --accent-cyan-hover: #0891b2;
  --accent-emerald: #10b981;
  --accent-rose: #f43f5e;
  --accent-amber: #f59e0b;
  --accent-purple: #a855f7;

  --shadow-card: 0 10px 25px -5px rgba(0, 0, 0, 0.5), 0 8px 10px -6px rgba(0, 0, 0, 0.4);
  --shadow-glow-cyan: 0 0 20px rgba(6, 182, 212, 0.25);
  --shadow-glow-indigo: 0 0 20px rgba(99, 102, 241, 0.25);
  --glass-blur: blur(12px);
}

[data-theme="light"] {
  --bg-primary: #f8fafc;
  --bg-surface: #ffffff;
  --bg-card: rgba(255, 255, 255, 0.85);
  --bg-card-hover: #ffffff;
  --bg-input: #f1f5f9;
  --border-subtle: rgba(0, 0, 0, 0.08);
  --border-accent: rgba(99, 102, 241, 0.4);

  --text-main: #0f172a;
  --text-muted: #475569;
  --text-dim: #94a3b8;

  --shadow-card: 0 10px 25px -5px rgba(0, 0, 0, 0.06), 0 8px 10px -6px rgba(0, 0, 0, 0.04);
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: var(--font-sans);
  background-color: var(--bg-primary);
```

```
  color: var(--text-main);
  min-height: 100vh;
  line-height: 1.5;
  overflow-x: hidden;
  -webkit-font-smoothing: antialiased;
}

/* Background CAD Engineering Grid Subtle Texture */
body::before {
  content: '';
  position: fixed;
  inset: 0;
  background-image:
    radial-gradient(rgba(99, 102, 241, 0.08) 1px, transparent 1px),
    radial-gradient(rgba(6, 182, 212, 0.05) 1px, transparent 1px);
  background-size: 32px 32px;
  background-position: 0 0, 16px 16px;
  pointer-events: none;
  z-index: 0;
}

#app-container {
  position: relative;
  z-index: 1;
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}

/* ================================================================================
   Header & Top Navigation
   ================================================================================ */
.app-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0.85rem 1.75rem;
  background: var(--bg-surface);
  border-bottom: 1px solid var(--border-subtle);
  backdrop-filter: var(--glass-blur);
  position: sticky;
  top: 0;
  z-index: 50;
}

.brand-wrapper {
  display: flex;
  align-items: center;
  gap: 0.85rem;
}

.brand-icon {
  width: 38px;
  height: 38px;
  background: linear-gradient(135deg, var(--accent-indigo), var(--accent-cyan));
  border-radius: 9px;
  display: flex;
  align-items: center;
  justify-content: center;
  color: white;
  box-shadow: var(--shadow-glow-cyan);
}

.brand-title {
  font-size: 1.15rem;
  font-weight: 700;
  letter-spacing: -0.02em;
  display: flex;
  align-items: center;
  gap: 0.5rem;
```

```
}

.brand-subtitle {
  font-size: 0.72rem;
  color: var(--text-muted);
  text-transform: uppercase;
  letter-spacing: 0.06em;
  font-family: var(--font-mono);
}

.header-actions {
  display: flex;
  align-items: center;
  gap: 0.75rem;
}

/* Badges */
.badge {
  display: inline-flex;
  align-items: center;
  gap: 0.35rem;
  padding: 0.25rem 0.65rem;
  font-size: 0.75rem;
  font-weight: 600;
  border-radius: 9999px;
  font-family: var(--font-mono);
  border: 1px solid transparent;
  transition: all 0.2s ease;
}

.badge-success {
  background: rgba(16, 185, 129, 0.15);
  color: #34d399;
  border-color: rgba(16, 185, 129, 0.3);
}

.badge-warning {
  background: rgba(245, 158, 11, 0.15);
  color: #fbbf24;
  border-color: rgba(245, 158, 11, 0.3);
}

.badge-danger {
  background: rgba(244, 63, 94, 0.15);
  color: #fb7185;
  border-color: rgba(244, 63, 94, 0.3);
}

.badge-info {
  background: rgba(6, 182, 212, 0.15);
  color: #38bdf8;
  border-color: rgba(6, 182, 212, 0.3);
}

/* Buttons */
.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 0.45rem;
  padding: 0.5rem 0.9rem;
  font-size: 0.82rem;
  font-weight: 500;
  border-radius: 7px;
  border: 1px solid var(--border-subtle);
  background: var(--bg-card);
  color: var(--text-main);
  cursor: pointer;
  transition: all 0.2s cubic-bezier(0.4, 0, 0.2, 1);
  text-decoration: none;
```

```
    user-select: none;
  }

  .btn:hover {
    background: var(--bg-card-hover);
    border-color: var(--accent-indigo);
    transform: translateY(-1px);
  }

  .btn-primary {
    background: linear-gradient(135deg, var(--accent-indigo), #4f46e5);
    color: white;
    border: none;
    box-shadow: 0 4px 12px rgba(99, 102, 241, 0.35);
  }

  .btn-primary:hover {
    background: linear-gradient(135deg, #4f46e5, #4338ca);
    box-shadow: 0 6px 16px rgba(99, 102, 241, 0.5);
  }

  .btn-cyan {
    background: linear-gradient(135deg, var(--accent-cyan), #0891b2);
    color: white;
    border: none;
  }

  .btn-danger {
    background: rgba(244, 63, 94, 0.15);
    color: #fb7185;
    border-color: rgba(244, 63, 94, 0.3);
  }

  .btn-danger:hover {
    background: rgba(244, 63, 94, 0.3);
  }

  .btn-icon {
    padding: 0.5rem;
    border-radius: 8px;
  }

  /* Primary Hero Action: Generate SFD & BMD Button */
  .btn-generate-all {
    width: 100%;
    padding: 0.85rem 1.25rem;
    font-size: 0.95rem;
    font-weight: 700;
    letter-spacing: 0.02em;
    background: linear-gradient(135deg, #06b6d4 0%, #6366f1 50%, #4f46e5 100%);
    color: white;
    border: 1px solid rgba(255, 255, 255, 0.2);
    border-radius: 10px;
    cursor: pointer;
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 0.65rem;
    box-shadow: 0 4px 15px rgba(6, 182, 212, 0.35), 0 0 25px rgba(99, 102, 241, 0.3);
    transition: all 0.25s cubic-bezier(0.4, 0, 0.2, 1);
    text-transform: uppercase;
    position: relative;
    overflow: hidden;
  }

  .btn-generate-all:hover {
    transform: translateY(-2px);
    box-shadow: 0 8px 25px rgba(6, 182, 212, 0.5), 0 0 35px rgba(99, 102, 241, 0.5);
    background: linear-gradient(135deg, #0891b2 0%, #4f46e5 50%, #4338ca 100%);
  }
```

```
275
276   .btn-generate-all:active {
277     transform: translateY(0);
278   }
279
280   .btn-generate-all::after {
281     content: '';
282     position: absolute;
283     top: -50%;
284     left: -50%;
285     width: 200%;
286     height: 200%;
287     background: linear-gradient(60deg, transparent, rgba(255, 255, 255, 0.15), transparent);
288     transform: rotate(30deg);
289     animation: shine 4s infinite;
290   }
291
292   @keyframes shine {
293     0% { transform: translateX(-100%) rotate(30deg); }
294     20%, 100% { transform: translateX(100%) rotate(30deg); }
295   }
296
297   /* Quick Add Form in Card */
298   .quick-add-strip {
299     background: var(--bg-surface);
300     border: 1px solid var(--border-subtle);
301     border-radius: 8px;
302     padding: 0.65rem;
303     margin-bottom: 0.85rem;
304   }
305
306   /* Floating Toast Notification */
307   .toast-alert {
308     position: fixed;
309     bottom: 24px;
310     right: 24px;
311     background: rgba(15, 23, 42, 0.95);
312     color: #f8fafc;
313     backdrop-filter: blur(12px);
314     border: 1px solid var(--accent-cyan);
315     padding: 0.75rem 1.25rem;
316     border-radius: 8px;
317     font-size: 0.85rem;
318     font-weight: 600;
319     box-shadow: 0 10px 30px rgba(0, 0, 0, 0.6), 0 0 20px rgba(6, 182, 212, 0.3);
320     display: flex;
321     align-items: center;
322     gap: 0.6rem;
323     z-index: 10000;
324     transform: translateY(100px);
325     opacity: 0;
326     transition: all 0.3s cubic-bezier(0.16, 1, 0.3, 1);
327     pointer-events: none;
328   }
329
330   .toast-alert.show {
331     transform: translateY(0);
332     opacity: 1;
333   }
334
335   /* ============================================================================
336      Main Split Layout: Controls (Left) vs Visuals & Diagnostics (Right)
337      ============================================================================ */
338   .main-layout {
339     display: grid;
340     grid-template-columns: 420px 1fr;
341     gap: 1.25rem;
342     padding: 1.25rem 1.75rem;
343     flex: 1;
344   }
```

```
@media (max-width: 1200px) {
  .main-layout {
    grid-template-columns: 1fr;
  }
}

/* Control Panel (Left Column) */
.controls-panel {
  display: flex;
  flex-direction: column;
  gap: 1.25rem;
}

/* Glassmorphic Cards */
.card {
  background: var(--bg-card);
  backdrop-filter: var(--glass-blur);
  border: 1px solid var(--border-subtle);
  border-radius: 12px;
  padding: 1.25rem;
  box-shadow: var(--shadow-card);
  transition: border-color 0.25s ease;
}

.card:hover {
  border-color: rgba(99, 102, 241, 0.3);
}

.card-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: 1rem;
  padding-bottom: 0.65rem;
  border-bottom: 1px solid var(--border-subtle);
}

.card-title {
  font-size: 0.88rem;
  font-weight: 600;
  letter-spacing: -0.01em;
  display: flex;
  align-items: center;
  gap: 0.5rem;
  color: var(--text-main);
}

/* Form Controls */
.form-group {
  margin-bottom: 0.9rem;
}

.form-group:last-child {
  margin-bottom: 0;
}

.form-label {
  display: flex;
  justify-content: space-between;
  align-items: center;
  font-size: 0.78rem;
  font-weight: 500;
  color: var(--text-muted);
  margin-bottom: 0.35rem;
}

.form-label .val-tag {
  font-family: var(--font-mono);
  font-weight: 600;
```

```
    color: var(--accent-cyan);
  }

  .input-row {
    display: flex;
    align-items: center;
    gap: 0.75rem;
  }

  .input-slider {
    flex: 1;
    -webkit-appearance: none;
    appearance: none;
    height: 6px;
    background: var(--bg-input);
    border-radius: 4px;
    outline: none;
    border: 1px solid var(--border-subtle);
  }

  .input-slider::-webkit-slider-thumb {
    -webkit-appearance: none;
    appearance: none;
    width: 16px;
    height: 16px;
    border-radius: 50%;
    background: var(--accent-indigo);
    cursor: pointer;
    border: 2px solid white;
    box-shadow: 0 0 8px rgba(99, 102, 241, 0.6);
    transition: transform 0.1s ease;
  }

  .input-slider::-webkit-slider-thumb:hover {
    transform: scale(1.15);
  }

  .input-number {
    width: 82px;
    padding: 0.4rem 0.5rem;
    background: var(--bg-input);
    border: 1px solid var(--border-subtle);
    border-radius: 6px;
    color: var(--text-main);
    font-family: var(--font-mono);
    font-size: 0.82rem;
    text-align: right;
    outline: none;
    transition: border-color 0.2s ease;
  }

  .input-number:focus {
    border-color: var(--accent-indigo);
    box-shadow: 0 0 0 2px rgba(99, 102, 241, 0.25);
  }

  .select-custom {
    width: 100%;
    padding: 0.5rem 0.75rem;
    background: var(--bg-input);
    border: 1px solid var(--border-subtle);
    border-radius: 6px;
    color: var(--text-main);
    font-size: 0.82rem;
    outline: none;
    cursor: pointer;
  }

  .select-custom:focus {
    border-color: var(--accent-indigo);
```

```
}

/* Tabs for Load Manager */
.tabs-header {
  display: flex;
  background: var(--bg-input);
  border-radius: 8px;
  padding: 3px;
  gap: 3px;
  margin-bottom: 0.85rem;
}

.tab-btn {
  flex: 1;
  padding: 0.45rem 0.5rem;
  font-size: 0.75rem;
  font-weight: 500;
  border: none;
  background: transparent;
  color: var(--text-muted);
  border-radius: 6px;
  cursor: pointer;
  transition: all 0.2s ease;
}

.tab-btn.active {
  background: var(--accent-indigo);
  color: white;
  font-weight: 600;
}

/* Load Items List */
.load-list {
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
  max-height: 220px;
  overflow-y: auto;
  padding-right: 4px;
}

.load-item {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0.5rem 0.75rem;
  background: var(--bg-surface);
  border: 1px solid var(--border-subtle);
  border-radius: 7px;
  font-size: 0.78rem;
  transition: border-color 0.2s ease;
}

.load-item:hover {
  border-color: var(--border-accent);
}

.load-meta {
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

.load-color-tag {
  width: 8px;
  height: 8px;
  border-radius: 50%;
}

.load-val {
```

```
    font-family: var(--font-mono);
    font-weight: 600;
    color: var(--text-main);
  }

  .load-actions {
    display: flex;
    gap: 0.35rem;
  }

  /* ================================================================================
     Visualizations & Charts Area (Right Column)
     ================================================================================ */
  .visuals-panel {
    display: flex;
    flex-direction: column;
    gap: 1.25rem;
  }

  /* Quick Metrics Ribbon */
  .metrics-ribbon {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(170px, 1fr));
    gap: 0.85rem;
  }

  .metric-card {
    background: var(--bg-card);
    backdrop-filter: var(--glass-blur);
    border: 1px solid var(--border-subtle);
    border-radius: 10px;
    padding: 0.85rem 1rem;
    display: flex;
    flex-direction: column;
    gap: 0.25rem;
  }

  .metric-label {
    font-size: 0.72rem;
    font-weight: 600;
    text-transform: uppercase;
    letter-spacing: 0.05em;
    color: var(--text-muted);
    display: flex;
    align-items: center;
    justify-content: space-between;
  }

  .metric-value {
    font-size: 1.35rem;
    font-weight: 700;
    font-family: var(--font-mono);
    color: var(--text-main);
  }

  .metric-sub {
    font-size: 0.72rem;
    color: var(--text-dim);
    font-family: var(--font-mono);
  }

  /* Diagrams Viewport Stack */
  .diagram-card {
    position: relative;
    background: var(--bg-card);
    backdrop-filter: var(--glass-blur);
    border: 1px solid var(--border-subtle);
    border-radius: 12px;
    overflow: hidden;
    box-shadow: var(--shadow-card);
```

```
}

.diagram-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0.65rem 1.25rem;
  background: rgba(15, 23, 42, 0.4);
  border-bottom: 1px solid var(--border-subtle);
}

.diagram-title {
  font-size: 0.82rem;
  font-weight: 600;
  letter-spacing: 0.02em;
  display: flex;
  align-items: center;
  gap: 0.5rem;
  color: var(--text-main);
}

.diagram-canvas-container {
  width: 100%;
  height: 190px;
  position: relative;
  cursor: crosshair;
}

.diagram-canvas-container.beam-container {
  height: 200px;
}

.diagram-canvas {
  width: 100%;
  height: 100%;
  display: block;
}

/* Floating Crosshair HUD */
#crosshair-hud {
  position: fixed;
  pointer-events: none;
  background: rgba(15, 23, 42, 0.95);
  backdrop-filter: blur(16px);
  border: 1px solid var(--accent-indigo);
  border-radius: 8px;
  padding: 0.5rem 0.75rem;
  box-shadow: 0 10px 25px rgba(0, 0, 0, 0.6), 0 0 15px rgba(99, 102, 241, 0.4);
  z-index: 100;
  display: none;
  flex-direction: column;
  gap: 0.2rem;
  font-family: var(--font-mono);
  font-size: 0.75rem;
  transform: translate(15px, -50%);
  transition: opacity 0.15s ease;
}

.hud-item {
  display: flex;
  justify-content: space-between;
  gap: 1rem;
}

.hud-label {
  color: var(--text-muted);
}

.hud-val {
  font-weight: 700;
```

```
    color: var(--text-main);
  }

  /* ================================================================================
     Equilibrium & Critical Points Diagnostics Table
     ================================================================================ */
  .diagnostics-grid {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 1.25rem;
  }

  @media (max-width: 900px) {
    .diagnostics-grid {
      grid-template-columns: 1fr;
    }
  }

  .math-steps-box {
    background: var(--bg-surface);
    border: 1px solid var(--border-subtle);
    border-radius: 8px;
    padding: 0.85rem;
    font-family: var(--font-mono);
    font-size: 0.78rem;
    line-height: 1.6;
    color: var(--text-muted);
    max-height: 220px;
    overflow-y: auto;
    white-space: pre-wrap;
  }

  .points-table {
    width: 100%;
    border-collapse: collapse;
    font-size: 0.78rem;
    font-family: var(--font-mono);
  }

  .points-table th {
    text-align: left;
    padding: 0.45rem 0.6rem;
    border-bottom: 1px solid var(--border-subtle);
    color: var(--text-muted);
    font-size: 0.7rem;
    text-transform: uppercase;
  }

  .points-table td {
    padding: 0.45rem 0.6rem;
    border-bottom: 1px solid rgba(255, 255, 255, 0.04);
    color: var(--text-main);
  }

  /* ================================================================================
     Modals
     ================================================================================ */
  .modal-overlay {
    position: fixed;
    inset: 0;
    background: rgba(0, 0, 0, 0.7);
    backdrop-filter: blur(8px);
    display: flex;
    align-items: center;
    justify-content: center;
    z-index: 1000;
    opacity: 0;
    pointer-events: none;
    transition: opacity 0.25s ease;
  }
```

```
765
766   .modal-overlay.active {
767     opacity: 1;
768     pointer-events: auto;
769   }
770
771   .modal-box {
772     background: var(--bg-surface);
773     border: 1px solid var(--border-subtle);
774     border-radius: 12px;
775     width: 90%;
776     max-width: 580px;
777     padding: 1.5rem;
778     box-shadow: 0 20px 40px rgba(0, 0, 0, 0.6);
779     transform: scale(0.95);
780     transition: transform 0.25s ease;
781   }
782
783   .modal-overlay.active .modal-box {
784     transform: scale(1);
785   }
786
787   .modal-header {
788     display: flex;
789     align-items: center;
790     justify-content: space-between;
791     margin-bottom: 1.25rem;
792   }
793
794   .modal-title {
795     font-size: 1rem;
796     font-weight: 700;
797     display: flex;
798     align-items: center;
799     gap: 0.5rem;
800   }
801
802   .modal-footer {
803     display: flex;
804     justify-content: flex-end;
805     gap: 0.75rem;
806     margin-top: 1.5rem;
807   }
808
809   /* Preset Cards Grid */
810   .presets-grid {
811     display: grid;
812     grid-template-columns: 1fr;
813     gap: 0.75rem;
814     max-height: 400px;
815     overflow-y: auto;
816     padding-right: 4px;
817   }
818
819   .preset-card {
820     background: var(--bg-card);
821     border: 1px solid var(--border-subtle);
822     border-radius: 8px;
823     padding: 0.85rem;
824     cursor: pointer;
825     transition: all 0.2s ease;
826     display: flex;
827     flex-direction: column;
828     gap: 0.35rem;
829   }
830
831   .preset-card:hover {
832     border-color: var(--accent-indigo);
833     background: var(--bg-card-hover);
834     transform: translateY(-2px);
```

```
}

.preset-title {
  font-size: 0.85rem;
  font-weight: 600;
  color: var(--text-main);
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.preset-desc {
  font-size: 0.75rem;
  color: var(--text-muted);
}

/* ================================================================================
   Print / Export Report Stylesheet
   ================================================================================ */
@media print {
  body {
    background: #ffffff !important;
    color: #000000 !important;
  }

  body::before {
    display: none !important;
  }

  .app-header,
  .controls-panel,
  .btn,
  .header-actions,
  #crosshair-hud {
    display: none !important;
  }

  .main-layout {
    grid-template-columns: 1fr !important;
    padding: 0 !important;
  }

  .card, .diagram-card {
    border: 1px solid #ccc !important;
    box-shadow: none !important;
    background: #ffffff !important;
    page-break-inside: avoid;
    margin-bottom: 20px;
  }

  .diagram-canvas-container {
    height: 180px !important;
  }

  .print-only-header {
    display: block !important;
    margin-bottom: 20px;
    border-bottom: 2px solid #000;
    padding-bottom: 10px;
  }
}

.print-only-header {
  display: none;
}

```

</> 6. User Interface & Dashboard Markup (index.html)

612 lines • HTML

Semantic HTML5 structure for control sidebar, metrics ribbon, diagram viewports, and interactive modals.

```
<!DOCTYPE html>
<html lang="en" data-theme="dark">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Structural Beam Analysis: SFD & BMD Diagnostics</title>

  <!-- Inter & JetBrains Mono Fonts -->
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&family=JetBrains+Mono:wght@4

  <!-- Lucide Icons -->
  <script src="https://unpkg.com/lucide@latest"></script>

  <!-- KaTeX for crisp LaTeX Equations rendering -->
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/katex@0.16.8/dist/katex.min.css">
  <script src="https://cdn.jsdelivr.net/npm/katex@0.16.8/dist/katex.min.js"></script>

  <!-- App Styles -->
  <link rel="stylesheet" href="styles.css">
</head>
<body>

  <!-- Floating Linked Crosshair HUD -->
  <div id="crosshair-hud">
    <div class="hud-item">
      <span class="hud-label">Position x:</span>
      <span class="hud-val" id="hud-x">0.00 m</span>
    </div>
    <div class="hud-item">
      <span class="hud-label">Shear V(x):</span>
      <span class="hud-val" id="hud-v" style="color: var(--accent-cyan);">0.00 kN</span>
    </div>
    <div class="hud-item">
      <span class="hud-label">Moment M(x):</span>
      <span class="hud-val" id="hud-m" style="color: var(--accent-indigo);">0.00 kN·m</span>
    </div>
    <div class="hud-item">
      <span class="hud-label">Stress σ(x):</span>
      <span class="hud-val" id="hud-stress" style="color: var(--accent-amber);">0.00 MPa</span>
    </div>
    <div class="hud-item">
      <span class="hud-label">Status:</span>
      <span class="hud-val" id="hud-status" style="color: var(--text-muted); font-size: 0.7rem;">Normal Span</span
    </div>
  </div>

  <div id="app-container">

    <!-- Top Header -->
    <header class="app-header">
      <div class="brand-wrapper">
        <div class="brand-icon">
          <i data-lucide="activity"></i>
        </div>
        <div>
          <h1 class="brand-title">
            BEAM DIAGNOSTICS <span class="badge badge-info">CAD v2.5</span>
          </h1>
          <p class="brand-subtitle">Shear Force & Bending Moment Analytical Engine</p>
        </div>
      </div>

```

```
        <div class="header-actions">
          <!-- Prominent Header Generate Button -->
          <button class="btn btn-primary" id="btn-header-generate" style="background: linear-gradient(135deg, var(--
            <i data-lucide="zap"></i> Generate SFD & BMD
          </button>

          <!-- Live Equilibrium Badge -->
          <span class="badge badge-success" id="equilibrium-badge">
            <i data-lucide="check-circle-2"></i> ∑Fy = 0.00 kN [PASS]
          </span>

          <!-- Quick Presets -->
          <button class="btn" id="btn-presets">
            <i data-lucide="layers"></i> Presets
          </button>

          <!-- Convention Toggle -->
          <button class="btn" id="btn-convention" title="Toggle Sagging / Hogging sign convention">
            <i data-lucide="sliders"></i> Sagging +
          </button>

          <!-- LaTeX Equations -->
          <button class="btn" id="btn-latex">
            <i data-lucide="binary"></i> LaTeX Math
          </button>

          <!-- Export CSV -->
          <button class="btn" id="btn-export-csv">
            <i data-lucide="download"></i> CSV
          </button>

          <!-- Export PDF / Print -->
          <button class="btn btn-primary" id="btn-export-print">
            <i data-lucide="printer"></i> Export Report
          </button>

          <!-- Dark/Light Theme Toggle -->
          <button class="btn btn-icon" id="btn-theme" title="Toggle Dark/Light Mode">
            <i data-lucide="moon" id="theme-icon"></i>
          </button>
        </div>
      </header>

      <!-- Floating Toast Notification -->
      <div id="toast-notification" class="toast-alert">
        <i data-lucide="check-circle" style="color: var(--accent-emerald);"></i>
        <span id="toast-msg">SFD & BMD Generated Successfully</span>
      </div>

      <!-- Print Only Header for Engineering Reports -->
      <div class="print-only-header">
        <h1 style="font-size: 1.5rem; font-weight: bold;">Structural Engineering Beam Diagnostic Report</h1>
        <p style="color: #666; font-size: 0.9rem;">Automated Section Mechanics & Static Equilibrium Verification</p>
        <div id="print-timestamp" style="margin-top: 5px; font-size: 0.8rem; font-family: monospace;"></div>
      </div>

      <!-- Main Content Layout -->
      <main class="main-layout">

        <!-- ================================================================
             LEFT COLUMN: CONTROLS PANEL
             ================================================================ -->
        <aside class="controls-panel">

          <!-- Primary Hero Action: Generate SFD & BMD Button -->
          <button class="btn-generate-all" id="btn-generate-diagrams" title="Click to calculate reactions and genera
            <i data-lucide="zap" style="width: 20px; height: 20px;"></i>
            <span>Generate SFD & BMD</span>
          </button>

```

```
          <!-- 1. Beam Geometry & Supports Card -->
          <section class="card">
            <div class="card-header">
              <h2 class="card-title">
                <i data-lucide="ruler"></i> Beam Configuration
              </h2>
              <span class="badge badge-info" id="span-info-badge">Span: 6.00 m</span>
            </div>

            <!-- Beam Length L -->
            <div class="form-group">
              <label class="form-label">
                <span>Beam Length (L)</span>
                <span class="val-tag" id="val-length">10.00 m</span>
              </label>
              <div class="input-row">
                <input type="range" class="input-slider" id="slider-length" min="2" max="30" step="0.5" value="10">
                <input type="number" class="input-number" id="num-length" min="1" max="50" step="0.1" value="10.0">
              </div>
            </div>

            <!-- Support System Mode -->
            <div class="form-group">
              <label class="form-label">Support Boundary Conditions</label>
              <select class="select-custom" id="select-support-system">
                <option value="two-supports">Two Supports (Simply Supported / Overhanging)</option>
                <option value="cantilever-left">Cantilever (Clamped at Left x = 0m)</option>
                <option value="cantilever-right">Cantilever (Clamped at Right x = L)</option>
              </select>
            </div>

            <!-- Support Positions Container (Hidden for cantilevers) -->
            <div id="two-supports-controls">
              <!-- Support A (Pin) -->
              <div class="form-group">
                <label class="form-label">
                  <span>Support A Position (Pin)</span>
                  <span class="val-tag" id="val-support-a">2.00 m</span>
                </label>
                <div class="input-row">
                  <input type="range" class="input-slider" id="slider-support-a" min="0" max="10" step="0.25" value=
                  <input type="number" class="input-number" id="num-support-a" min="0" max="10" step="0.1" value="2.
                </div>
              </div>

              <!-- Support B (Roller) -->
              <div class="form-group">
                <label class="form-label">
                  <span>Support B Position (Roller)</span>
                  <span class="val-tag" id="val-support-b">8.00 m</span>
                </label>
                <div class="input-row">
                  <input type="range" class="input-slider" id="slider-support-b" min="0" max="10" step="0.25" value=
                  <input type="number" class="input-number" id="num-support-b" min="0" max="10" step="0.1" value="8.
                </div>
              </div>

              <!-- Overhang breakdown pill indicators -->
              <div style="display: flex; gap: 0.4rem; margin-top: 0.6rem;">
                <span class="badge" id="overhang-left-pill" style="font-size: 0.7rem; flex: 1; justify-content: cent
                <span class="badge" id="overhang-right-pill" style="font-size: 0.7rem; flex: 1; justify-content: cen
              </div>
            </div>
          </section>

          <!-- 2. Dynamic Load Customizer Card -->
          <section class="card">
            <div class="card-header">
              <h2 class="card-title">
                <i data-lucide="weight"></i> Load Customizer
```

```
            </h2>
            <button class="btn btn-cyan" id="btn-add-load-modal" style="padding: 0.35rem 0.65rem; font-size: 0.75r
              <i data-lucide="plus"></i> Add Load
            </button>
          </div>

          <!-- Quick Add Inline Form -->
          <div class="quick-add-strip">
            <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 0.4rem;
              <span style="font-size: 0.72rem; font-weight: 600; color: var(--text-muted); text-transform: upperca
              <span style="font-size: 0.68rem; color: var(--accent-cyan);">Instant Setup</span>
            </div>
            <div style="display: grid; grid-template-columns: 85px 1fr 1fr auto; gap: 0.35rem; align-items: center
              <select class="select-custom" id="quick-load-type" style="padding: 0.35rem 0.3rem; font-size: 0.75re
                <option value="point">Point (P)</option>
                <option value="udl">UDL (w)</option>
                <option value="moment">Moment (M)</option>
              </select>
              <input type="number" class="input-number" id="quick-load-x" placeholder="x (m)" value="5.0" step="0.
              <input type="number" class="input-number" id="quick-load-mag" placeholder="Mag (kN)" value="30" step
              <button class="btn btn-cyan" id="btn-quick-add" style="padding: 0.35rem 0.65rem; font-size: 0.75rem;
                <i data-lucide="plus"></i> Add
              </button>
            </div>
          </div>

          <!-- Tabs for load categories -->
          <div class="tabs-header">
            <button class="tab-btn active" data-tab="all">All Loads (<span id="count-all">0</span>)</button>
            <button class="tab-btn" data-tab="point">Point (<span id="count-point">0</span>)</button>
            <button class="tab-btn" data-tab="udl">UDL (<span id="count-udl">0</span>)</button>
            <button class="tab-btn" data-tab="moment">Moment (<span id="count-moment">0</span>)</button>
          </div>

          <!-- Live Load Items List -->
          <div class="load-list" id="loads-container">
            <!-- Dynamic load items rendered here -->
          </div>

          <!-- Button to Calculate & Plot Diagrams -->
          <button class="btn btn-primary" id="btn-calculate-loads" style="width: 100%; margin-top: 0.75rem; font-w
            <i data-lucide="play"></i> Calculate & Plot SFD / BMD
          </button>
        </section>

        <!-- 3. Cross-Section & Stress Diagnostics Card -->
        <section class="card">
          <div class="card-header">
            <h2 class="card-title">
              <i data-lucide="shield-alert"></i> Cross-Section & Stress
            </h2>
            <span class="badge badge-success" id="stress-badge">Safe: 45.2%</span>
          </div>

          <div class="form-group">
            <label class="form-label">Beam Profile Geometry</label>
            <select class="select-custom" id="select-profile-type">
              <option value="custom">Direct Section Modulus (Z)</option>
              <option value="rect">Rectangular (b × h)</option>
              <option value="circle">Solid Circular (Diameter d)</option>
              <option value="i-beam">Standard Steel I-Beam (IPE 300 / W12x26)</option>
            </select>
          </div>

          <!-- Rectangular Inputs -->
          <div id="rect-inputs" style="display: none; margin-bottom: 0.75rem;">
            <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 0.5rem;">
              <div>
                <label class="form-label">Width b (mm)</label>
                <input type="number" class="input-number" id="rect-b" value="150" style="width: 100%;">
```

```
                </div>
                <div>
                  <label class="form-label">Height h (mm)</label>
                  <input type="number" class="input-number" id="rect-h" value="300" style="width: 100%;">
                </div>
              </div>
            </div>

            <!-- Section Modulus Z -->
            <div class="form-group">
              <label class="form-label">
                <span>Elastic Section Modulus (Z)</span>
                <span class="val-tag" id="val-section-modulus">500 cm³</span>
              </label>
              <div class="input-row">
                <input type="range" class="input-slider" id="slider-section-modulus" min="50" max="4000" step="25" v
                <input type="number" class="input-number" id="num-section-modulus" min="10" max="20000" step="10" va
              </div>
            </div>

            <!-- Allowable Stress -->
            <div class="form-group">
              <label class="form-label">
                <span>Material Yield / Allowable Stress (σ_allow)</span>
                <span class="val-tag" id="val-allowable-stress">250 MPa</span>
              </label>
              <select class="select-custom" id="select-material" style="margin-bottom: 0.5rem;">
                <option value="250">Structural Steel S275 / A36 (250 MPa)</option>
                <option value="355">High-Strength Steel S355 (355 MPa)</option>
                <option value="120">Structural Aluminum 6061-T6 (120 MPa)</option>
                <option value="18">Structural Glulam Timber (18 MPa)</option>
                <option value="custom">Custom Value...</option>
              </select>
              <div class="input-row">
                <input type="range" class="input-slider" id="slider-allowable-stress" min="10" max="600" step="5" va
                <input type="number" class="input-number" id="num-allowable-stress" min="5" max="1000" step="5" valu
              </div>
            </div>
          </section>

        </aside>

        <!-- ====================================================================
             RIGHT COLUMN: REAL-TIME VISUALIZATIONS & DIAGNOSTICS
             ==================================================================== -->
        <section class="visuals-panel">

          <!-- Engineering Summary Metrics Ribbon -->
          <div class="metrics-ribbon">
            <!-- Equilibrium Balance -->
            <div class="metric-card">
              <div class="metric-label">
                <span>Equilibrium Balance</span>
                <i data-lucide="scale" style="width: 14px; height: 14px; color: var(--accent-emerald);"></i>
              </div>
              <div class="metric-value" id="metric-equilibrium" style="color: var(--accent-emerald);">Δ = 0.00 kN</d
              <div class="metric-sub" id="metric-load-sum">∑F_down: 85.0 kN</div>
            </div>

            <!-- Reactions -->
            <div class="metric-card">
              <div class="metric-label">
                <span>Support Reactions</span>
                <i data-lucide="arrow-up" style="width: 14px; height: 14px; color: var(--accent-cyan);"></i>
              </div>
              <div class="metric-value" id="metric-reactions" style="font-size: 1.1rem; color: var(--accent-cyan);">
              <div class="metric-sub" id="metric-reactions-sub">Total Reaction: 85.0 kN</div>
            </div>

            <!-- Peak Moment M_max -->
```

```
            <div class="metric-card">
              <div class="metric-label">
                <span>Max Bending Moment</span>
                <i data-lucide="trending-up" style="width: 14px; height: 14px; color: var(--accent-indigo);"></i>
              </div>
              <div class="metric-value" id="metric-max-moment" style="color: var(--accent-indigo);">85.0 kN·m</div>
              <div class="metric-sub" id="metric-moment-pos">At x = 6.00 m (Sagging)</div>
            </div>

            <!-- Peak Shear V_max -->
            <div class="metric-card">
              <div class="metric-label">
                <span>Max Shear Force</span>
                <i data-lucide="git-commit" style="width: 14px; height: 14px; color: var(--accent-amber);"></i>
              </div>
              <div class="metric-value" id="metric-max-shear" style="color: var(--accent-amber);">42.5 kN</div>
              <div class="metric-sub" id="metric-shear-pos">At x = 2.00 m</div>
            </div>

            <!-- Peak Bending Stress -->
            <div class="metric-card">
              <div class="metric-label">
                <span>Peak Stress (σ = M/Z)</span>
                <i data-lucide="zap" style="width: 14px; height: 14px; color: var(--accent-rose);"></i>
              </div>
              <div class="metric-value" id="metric-stress">170.0 MPa</div>
              <div class="metric-sub" id="metric-safety-factor">Factor of Safety: 1.47</div>
            </div>
          </div>

          <!-- 1. Schematic Beam View Diagram -->
          <div class="diagram-card">
            <div class="diagram-header">
              <div class="diagram-title">
                <i data-lucide="git-merge" style="color: var(--accent-cyan);"></i>
                <span>2D Beam Schematic & Free-Body Loading</span>
              </div>
              <span class="badge badge-info" id="schematic-badge">Length: 10.0m</span>
            </div>
            <div class="diagram-canvas-container beam-container">
              <canvas id="canvas-beam" class="diagram-canvas"></canvas>
            </div>
          </div>

          <!-- 2. Shear Force Diagram (SFD) -->
          <div class="diagram-card">
            <div class="diagram-header">
              <div class="diagram-title">
                <i data-lucide="activity" style="color: var(--accent-cyan);"></i>
                <span>Shear Force Diagram (SFD) [kN]</span>
              </div>
              <span class="badge badge-warning" id="zero-shear-badge">Zero Shear @ x = 6.00 m</span>
            </div>
            <div class="diagram-canvas-container">
              <canvas id="canvas-sfd" class="diagram-canvas"></canvas>
            </div>
          </div>

          <!-- 3. Bending Moment Diagram (BMD) -->
          <div class="diagram-card">
            <div class="diagram-header">
              <div class="diagram-title">
                <i data-lucide="pie-chart" style="color: var(--accent-indigo);"></i>
                <span>Bending Moment Diagram (BMD) [kN·m]</span>
              </div>
              <span class="badge badge-info" id="contraflexure-badge">2 Inflection Points</span>
            </div>
            <div class="diagram-canvas-container">
              <canvas id="canvas-bmd" class="diagram-canvas"></canvas>
            </div>
```

```
          </div>

          <!-- Diagnostics & Equilibrium Step-by-Step Math Breakdown -->
          <div class="diagnostics-grid">

            <!-- Static Equilibrium Verification -->
            <div class="card">
              <div class="card-header">
                <h3 class="card-title">
                  <i data-lucide="file-text"></i> Equilibrium Math Derivation
                </h3>
                <span class="badge badge-success">Statically Determinate</span>
              </div>
              <div class="math-steps-box" id="equilibrium-steps-content">
                Loading equilibrium steps...
              </div>
            </div>

            <!-- Critical Points & Section Table -->
            <div class="card">
              <div class="card-header">
                <h3 class="card-title">
                  <i data-lucide="list"></i> Critical Section Points
                </h3>
                <span class="badge badge-info" id="crit-points-count">5 Points</span>
              </div>
              <div style="max-height: 220px; overflow-y: auto;">
                <table class="points-table">
                  <thead>
                    <tr>
                      <th>x (m)</th>
                      <th>V (kN)</th>
                      <th>M (kN·m)</th>
                      <th>Classification</th>
                    </tr>
                  </thead>
                  <tbody id="critical-points-table-body">
                    <!-- Dynamic rows -->
                  </tbody>
                </table>
              </div>
            </div>

          </div>

        </section>

      </main>

    </div>

    <!-- ========================================================================
         MODAL: ADD LOAD MODAL
         ======================================================================== -->
    <div class="modal-overlay" id="modal-add-load">
      <div class="modal-box">
        <div class="modal-header">
          <h3 class="modal-title">
            <i data-lucide="plus-circle" style="color: var(--accent-cyan);"></i> Add Applied Load
          </h3>
          <button class="btn btn-icon btn-modal-close"><i data-lucide="x"></i></button>
        </div>

        <div class="form-group">
          <label class="form-label">Load Type</label>
          <select class="select-custom" id="modal-load-type">
            <option value="point">Concentrated / Point Load (kN)</option>
            <option value="udl">Uniformly Distributed Load - UDL (kN/m)</option>
            <option value="moment">Concentrated Moment (kN·m)</option>
          </select>
```

```
        </div>

        <!-- Point Load Fields -->
        <div id="modal-fields-point">
          <div class="form-group">
            <label class="form-label">Position x (m)</label>
            <input type="number" class="input-number" id="modal-pl-x" value="5.0" min="0" step="0.1" style="width: 1
          </div>
          <div class="form-group">
            <label class="form-label">Magnitude (kN)</label>
            <input type="number" class="input-number" id="modal-pl-mag" value="30" step="1" style="width: 100%;">
          </div>
          <div class="form-group">
            <label class="form-label">Direction</label>
            <select class="select-custom" id="modal-pl-dir">
              <option value="down">Downward (Standard Gravity Load ↓)</option>
              <option value="up">Upward (Wind / Uplift Force ↑)</option>
            </select>
          </div>
        </div>

        <!-- UDL Fields -->
        <div id="modal-fields-udl" style="display: none;">
          <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 0.75rem; margin-bottom: 0.9rem;">
            <div>
              <label class="form-label">Start Position x1 (m)</label>
              <input type="number" class="input-number" id="modal-udl-x1" value="2.0" min="0" step="0.1" style="widt
            </div>
            <div>
              <label class="form-label">End Position x2 (m)</label>
              <input type="number" class="input-number" id="modal-udl-x2" value="8.0" min="0" step="0.1" style="widt
            </div>
          </div>
          <div class="form-group">
            <label class="form-label">Uniform Intensity w (kN/m)</label>
            <input type="number" class="input-number" id="modal-udl-mag" value="15" step="1" style="width: 100%;">
          </div>
        </div>

        <!-- Moment Fields -->
        <div id="modal-fields-moment" style="display: none;">
          <div class="form-group">
            <label class="form-label">Position x (m)</label>
            <input type="number" class="input-number" id="modal-moment-x" value="4.0" min="0" step="0.1" style="widt
          </div>
          <div class="form-group">
            <label class="form-label">Magnitude (kN·m)</label>
            <input type="number" class="input-number" id="modal-moment-mag" value="25" step="1" style="width: 100%;"
          </div>
          <div class="form-group">
            <label class="form-label">Sense of Rotation</label>
            <select class="select-custom" id="modal-moment-dir">
              <option value="cw">Clockwise ↻</option>
              <option value="ccw">Counter-Clockwise ↺</option>
            </select>
          </div>
        </div>

        <div class="modal-footer">
          <button class="btn btn-modal-close">Cancel</button>
          <button class="btn btn-primary" id="btn-save-new-load">Add Load</button>
        </div>
      </div>
    </div>

    <!-- ========================================================================
         MODAL: PRESETS MODAL
         ======================================================================== -->
    <div class="modal-overlay" id="modal-presets">
      <div class="modal-box" style="max-width: 680px;">
```

```
        <div class="modal-header">
          <h3 class="modal-title">
            <i data-lucide="layers" style="color: var(--accent-indigo);"></i> Loading & Beam Presets
          </h3>
          <button class="btn btn-icon btn-modal-close"><i data-lucide="x"></i></button>
        </div>

        <div class="presets-grid" id="presets-list-container">
          <!-- Preset cards rendered dynamically -->
        </div>

        <div class="modal-footer">
          <button class="btn btn-modal-close">Close</button>
        </div>
      </div>
    </div>

    <!-- ========================================================================
         MODAL: LATEX EQUATIONS MODAL
         ======================================================================== -->
    <div class="modal-overlay" id="modal-latex">
      <div class="modal-box" style="max-width: 650px;">
        <div class="modal-header">
          <h3 class="modal-title">
            <i data-lucide="binary" style="color: var(--accent-cyan);"></i> Piece-wise Analytical Equations
          </h3>
          <button class="btn btn-icon btn-modal-close"><i data-lucide="x"></i></button>
        </div>

        <div style="margin-bottom: 1rem;">
          <h4 style="font-size: 0.8rem; color: var(--text-muted); margin-bottom: 0.4rem;">Reactions:</h4>
          <div id="latex-reactions-render" style="padding: 0.6rem; background: var(--bg-surface); border-radius: 6px
        </div>

        <div style="margin-bottom: 1rem;">
          <h4 style="font-size: 0.8rem; color: var(--text-muted); margin-bottom: 0.4rem;">Shear Force V(x):</h4>
          <div id="latex-shear-render" style="padding: 0.6rem; background: var(--bg-surface); border-radius: 6px; ov
        </div>

        <div style="margin-bottom: 1.25rem;">
          <h4 style="font-size: 0.8rem; color: var(--text-muted); margin-bottom: 0.4rem;">Bending Moment M(x):</h4>
          <div id="latex-moment-render" style="padding: 0.6rem; background: var(--bg-surface); border-radius: 6px; o
        </div>

        <div class="modal-footer">
          <button class="btn btn-cyan" id="btn-copy-latex">
            <i data-lucide="copy"></i> Copy LaTeX
          </button>
          <button class="btn btn-modal-close">Close</button>
        </div>
      </div>
    </div>

    <!-- Main JavaScript App Modules -->
    <script type="module" src="app.js"></script>
  </body>
  </html>

```